What questions do you have about Fall 2026 classes?

yellkey.com/end

DUTCH 1

Elementary Dutch

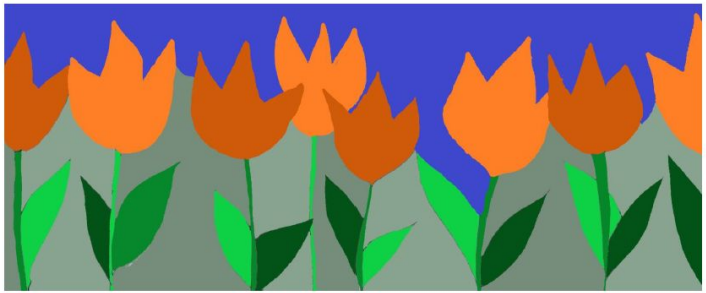


Image: ievanderhoeven

Interested in learning an easy language in between English and German? Consider taking Dutch 1! In this beginner's course, you will familiarize yourself with the basics of Dutch: its sounds and spelling, its grammatical structure, and its vocabulary. The class focuses on oral communication with an emphasis on vocabulary: Learning words and learning how to use these words. By reading texts and dialogues (and listening to the audio version), you will build your vocabulary. In class you will get the opportunity to practice your newly learned words and phrases. By the end of the semester, you will be able to express yourself in speaking and writing about a variety of topics, including introducing yourself, talking about your day, getting around, food and drinks, your house, and more.

M-W 12-2pm + F 12-1pm | 5 units

This course will be offered in person on campus and on Zoom.
Students from other UCs can enroll via UC Online.

For more information, please contact the instructor:

Esmée van der Hoeven, ievanderhoeven@berkeley.edu

Or visit: **dutch.berkeley.edu**

Fall 2026

Lecture 33 (Sorting 2)

Insertion Sort and Quicksort

CS61B, Spring 2026 @ UC Berkeley

Josh Hug and Kay Ousterhout

Naive Insertion Sort

Lecture 33, CS61B, Spring 2026

Insertion Sort

- **Naive Insertion Sort**
- In-Place Insertion Sort
- Insertion Sort Runtime

Quicksort

- Quicksort Backstory, Partitioning
- Quicksort

Insertion Sort

General strategy:

- Starting with an empty output sequence.
- Add each item from input, inserting into output at right point.

Naive approach, build entirely new output:

Input:

Output:

Insertion Sort

General strategy:

- Starting with an empty output sequence.
- Add each item from input, inserting into output at right point.

Naive approach, build entirely new output:

Input:

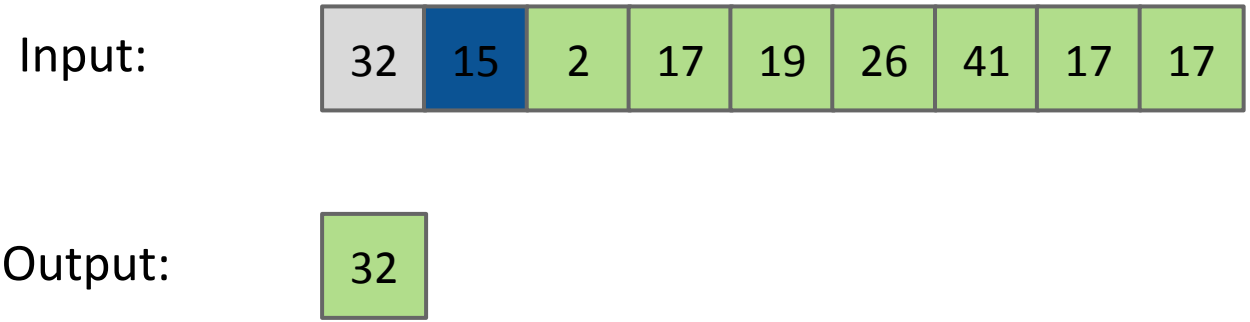
Output:

Insertion Sort

General strategy:

- Starting with an empty output sequence.
- Add each item from input, inserting into output at right point.

Naive approach, build entirely new output:

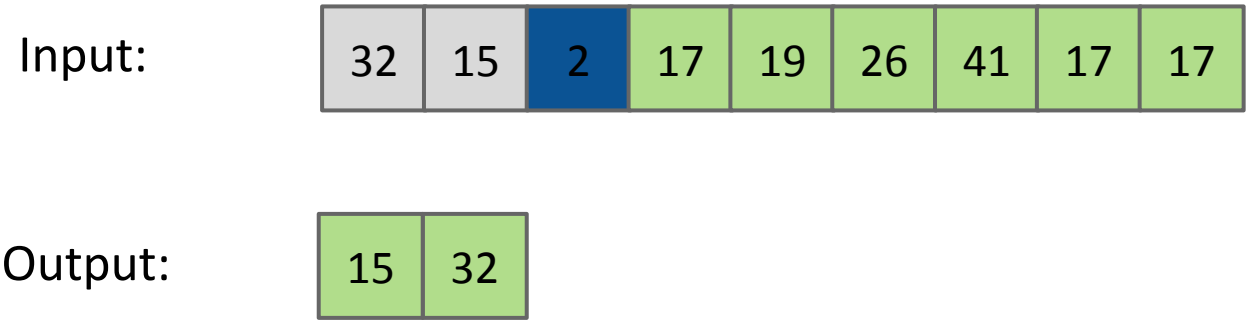


Insertion Sort

General strategy:

- Starting with an empty output sequence.
- Add each item from input, inserting into output at right point.

Naive approach, build entirely new output:

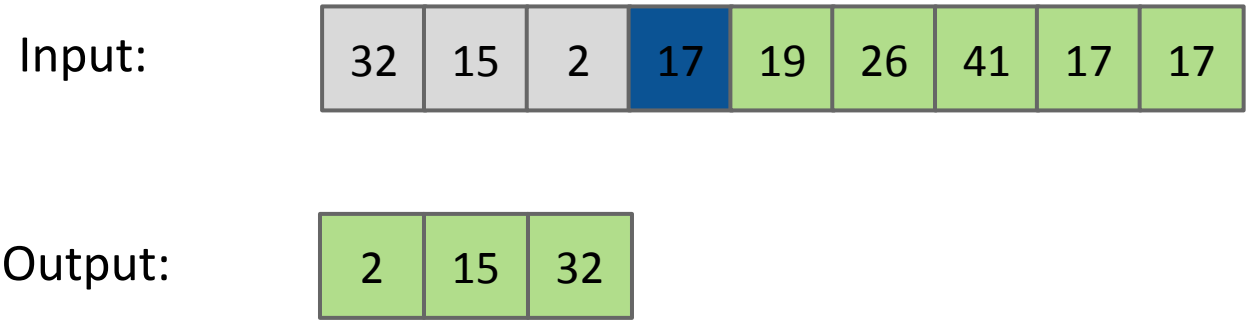


Insertion Sort

General strategy:

- Starting with an empty output sequence.
- Add each item from input, inserting into output at right point.

Naive approach, build entirely new output:

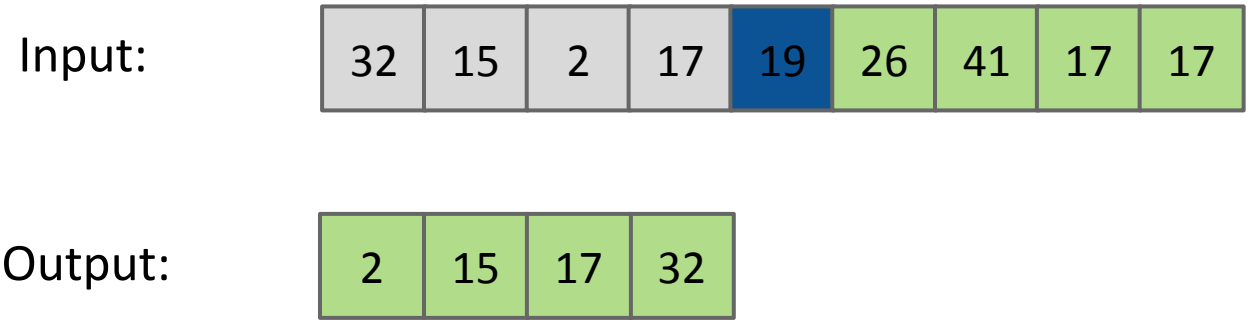


Insertion Sort

General strategy:

- Starting with an empty output sequence.
- Add each item from input, inserting into output at right point.

Naive approach, build entirely new output:

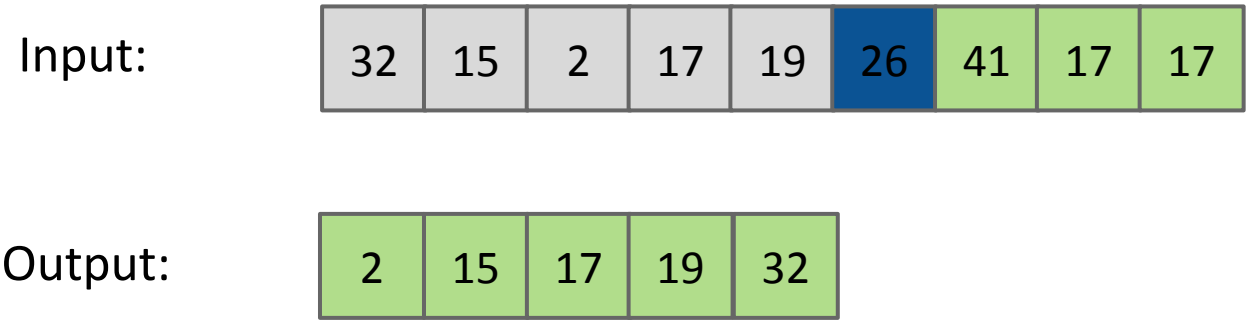


Insertion Sort

General strategy:

- Starting with an empty output sequence.
- Add each item from input, inserting into output at right point.

Naive approach, build entirely new output:

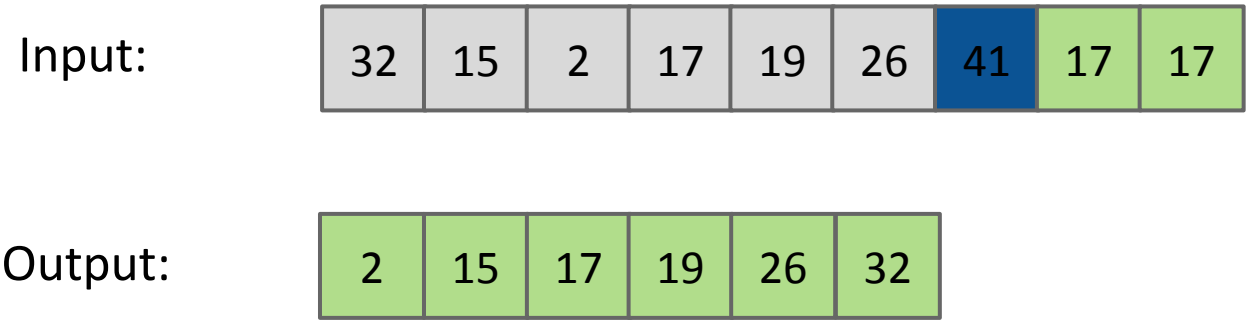


Insertion Sort

General strategy:

- Starting with an empty output sequence.
- Add each item from input, inserting into output at right point.

Naive approach, build entirely new output:

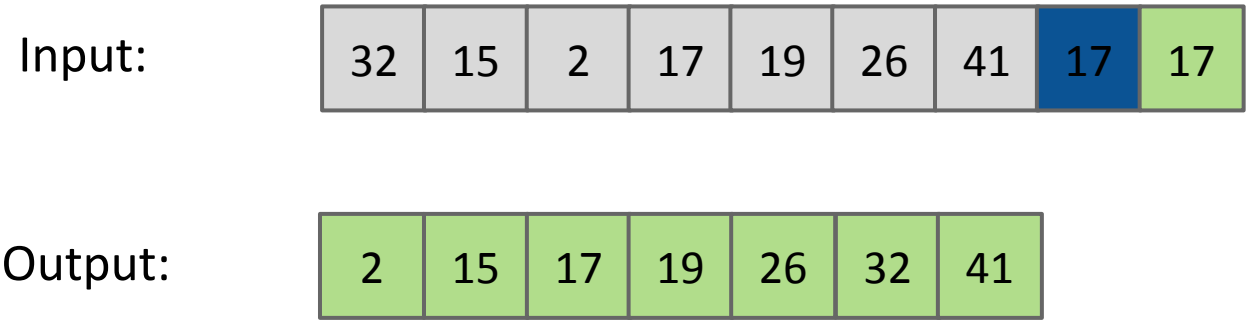


Insertion Sort

General strategy:

- Starting with an empty output sequence.
- Add each item from input, inserting into output at right point.

Naive approach, build entirely new output:

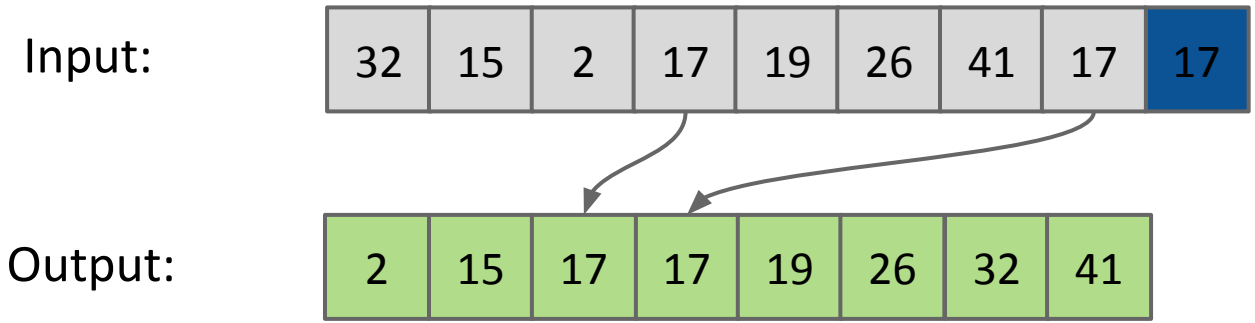


Insertion Sort

General strategy:

- Starting with an empty output sequence.
- Add each item from input, inserting into output at right point.

Naive approach, build entirely new output:

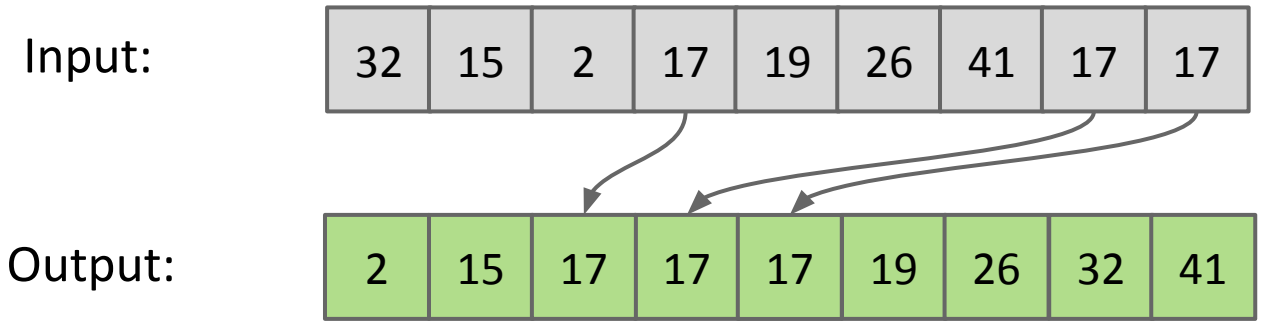


Insertion Sort

General strategy:

- Starting with an empty output sequence.
- Add each item from input, inserting into output at right point.

Naive approach, build entirely new output:



Can we eliminate this output array?

In-Place Insertion Sort

Lecture 33, CS61B, Spring 2026

Insertion Sort

- Naive Insertion Sort
- **In-Place Insertion Sort**
- Insertion Sort Runtime

Quicksort

- Quicksort Backstory, Partitioning
- Quicksort

Insertion Sort

General strategy:

- Starting with an empty output sequence.
- Add each item from input, inserting into output at right point.

For naive approach, if output sequence contains k items, worst cost to insert a single item is k.

- Might need to move everything over.

More efficient method:

- Do everything in place using swapping.

Insertion Sort

General strategy:

- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items

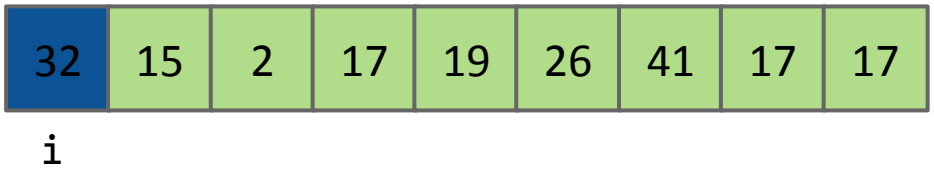
Input:

Insertion Sort

General strategy:

- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items

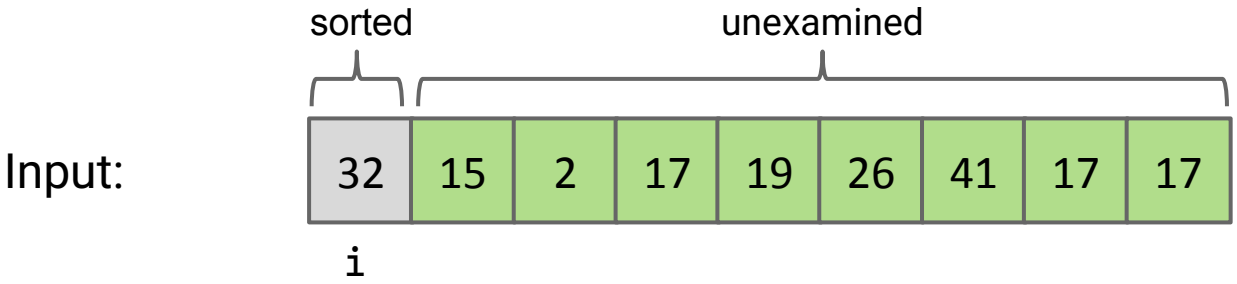
Input:



In-place Insertion Sort

General strategy:

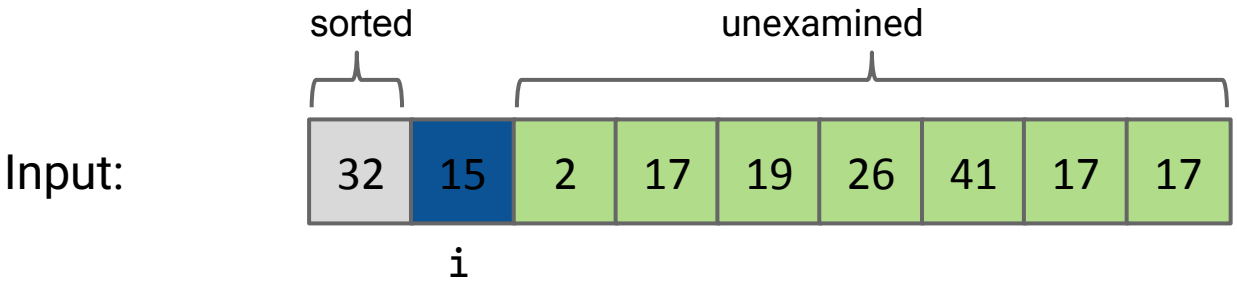
- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items



In-place Insertion Sort

General strategy:

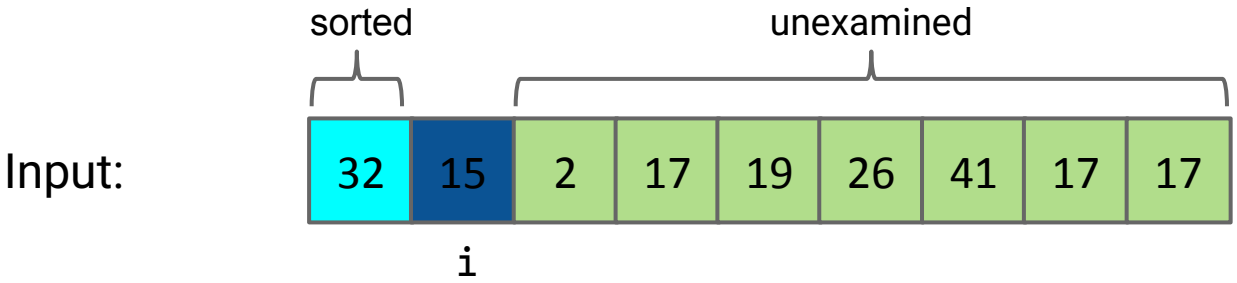
- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items



In-place Insertion Sort

General strategy:

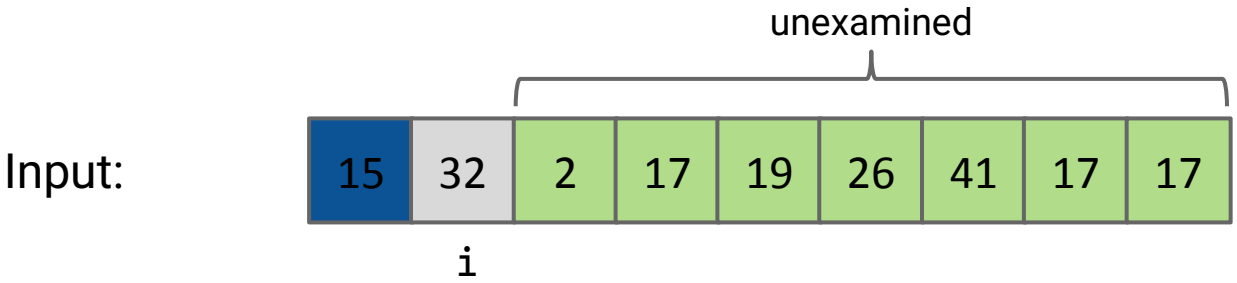
- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items (**traveller**)



In-place Insertion Sort

General strategy:

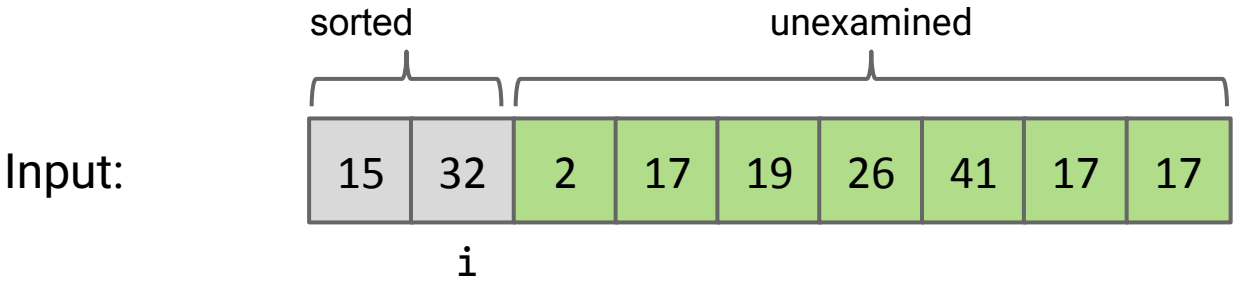
- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.



In-place Insertion Sort

General strategy:

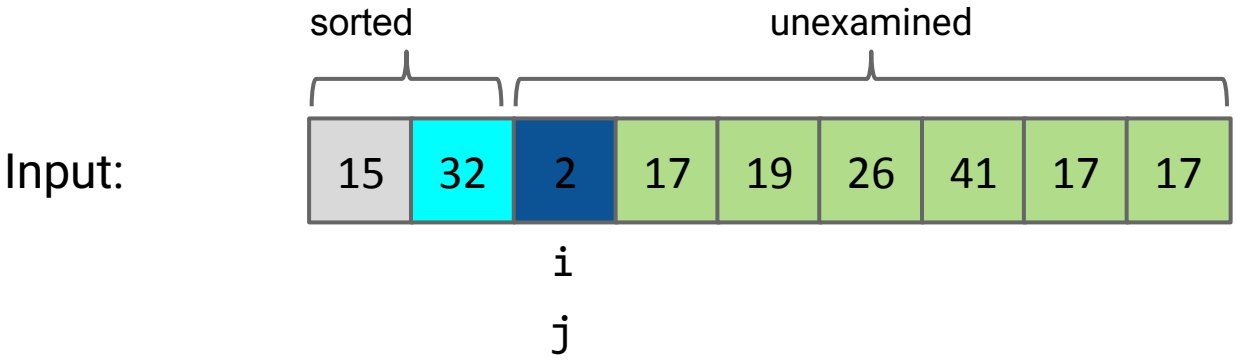
- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.



In-place Insertion Sort

General strategy:

- Repeat for i = 0 to N - 1:
  - "Insert" item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.

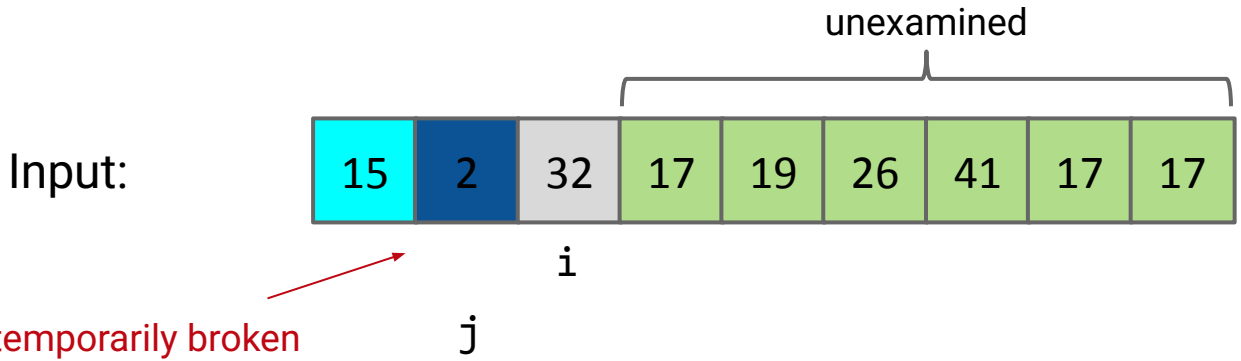


Use `j` pointer to track current spot of traveling item.

In-place Insertion Sort

General strategy:

- Repeat for i = 0 to N - 1:
  - "Insert" item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.



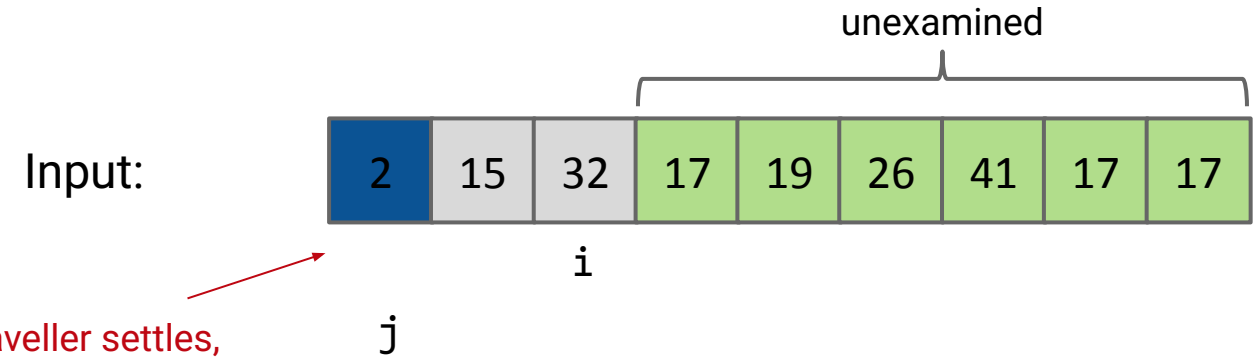
Note: We've temporarily broken our invariant that the items up through item i should be sorted!

In example above: Use j pointer to track current spot of traveling item.

In-place Insertion Sort

General strategy:

- Repeat for i = 0 to N - 1:
  - "Insert" item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.

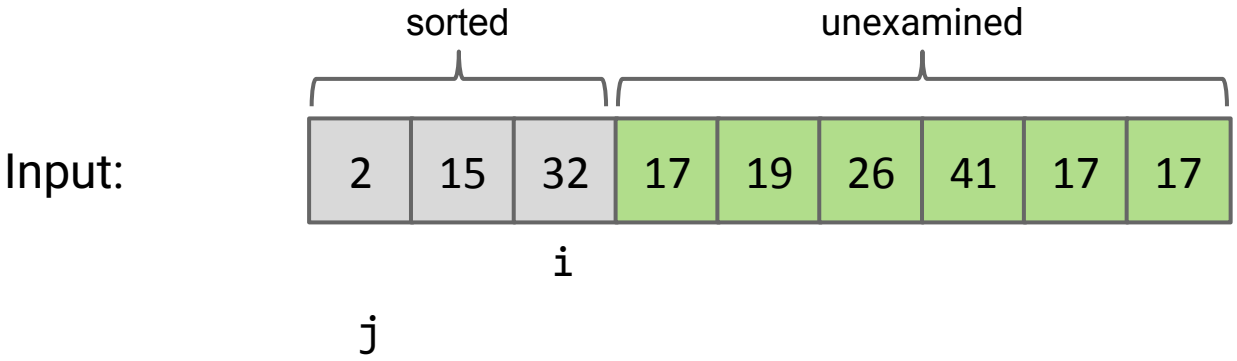


Use `j` pointer to track current spot of traveling item.

In-place Insertion Sort

General strategy:

- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.

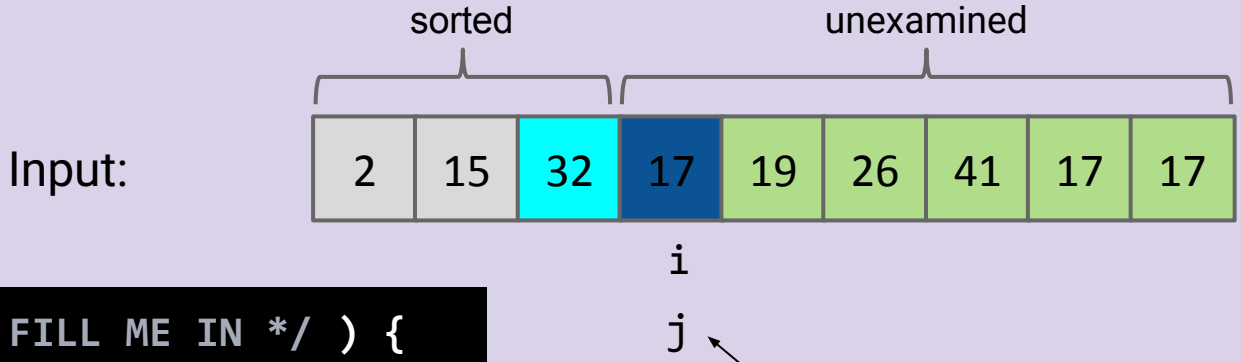


Use `j` pointer to track current spot of traveling item.

In-place Insertion Sort: yellkey.com/service

General strategy:

- Repeat for i = 0 to N - 1:
  - "Insert" item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.

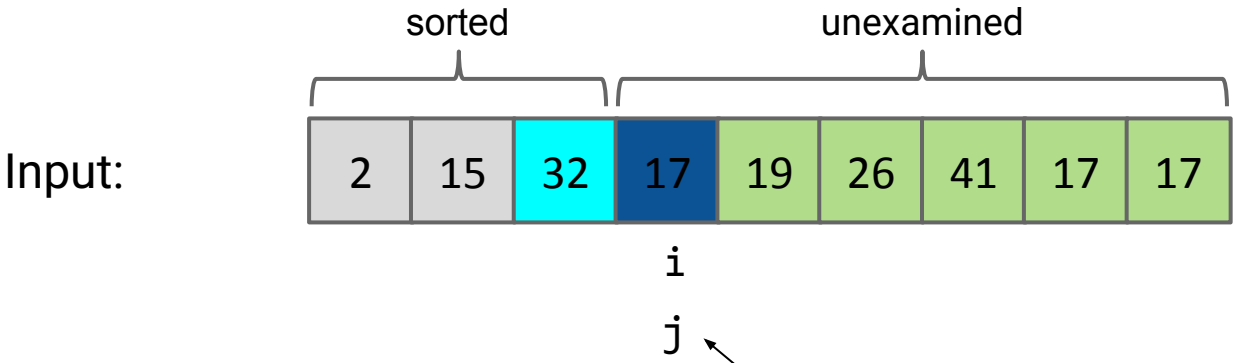



```
while (/* FILL ME IN */ ) {
   // Swap backwards
}
```

In-place Insertion Sort

General strategy:

- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.

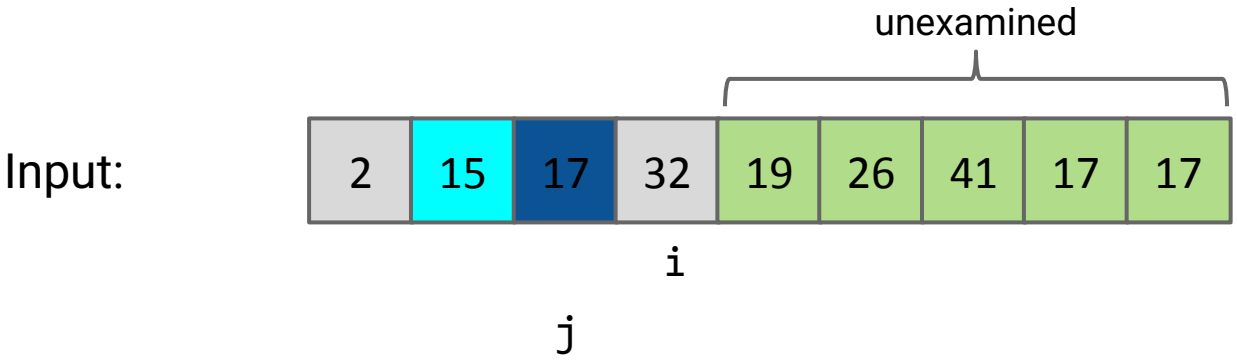



```
while (j > 0 && input[j - 1] > input[j]) {
   // Swap backwards
}
```

In-place Insertion Sort

General strategy:

- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.

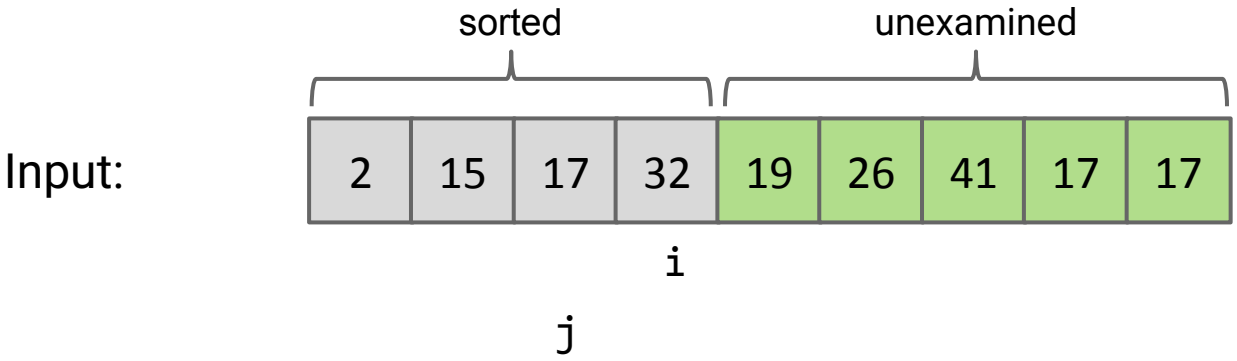


Use `j` pointer to track current spot of traveling item.

In-place Insertion Sort

General strategy:

- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.

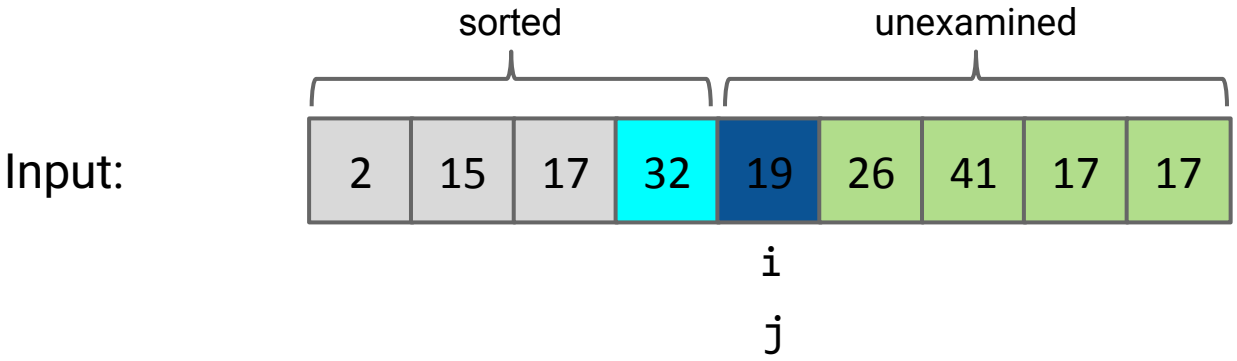


Use `j` pointer to track current spot of traveling item.

In-place Insertion Sort

General strategy:

- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.

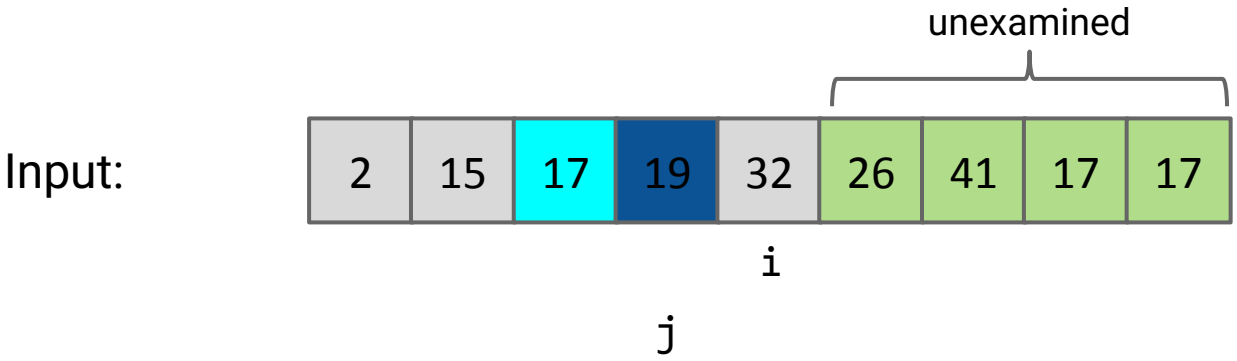


Use `j` pointer to track current spot of traveling item.

In-place Insertion Sort

General strategy:

- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.

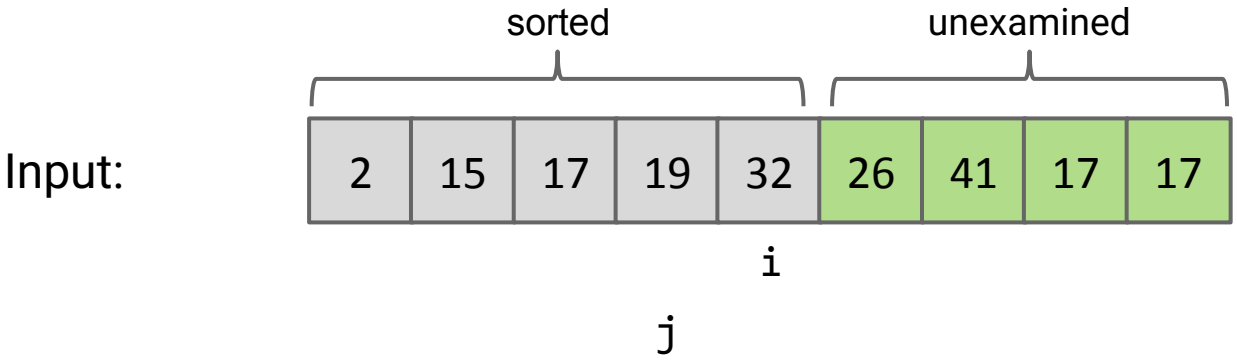


Use `j` pointer to track current spot of traveling item.

In-place Insertion Sort

General strategy:

- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.

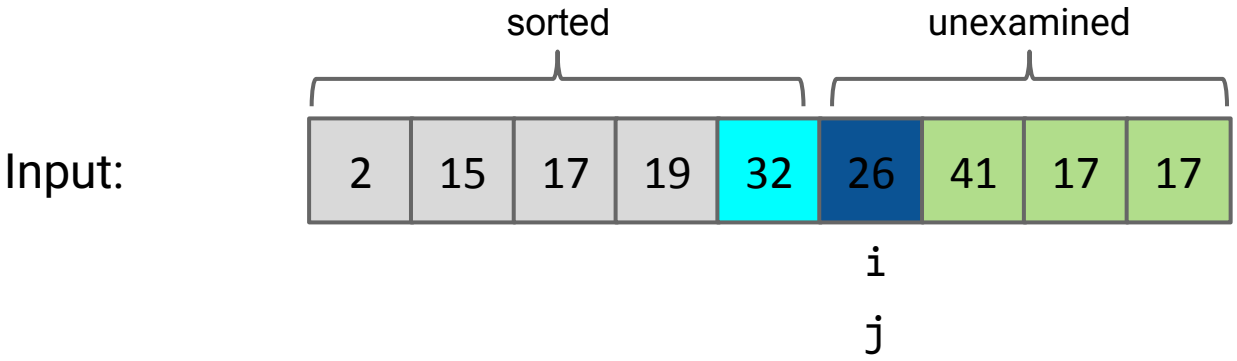


Use `j` pointer to track current spot of traveling item.

In-place Insertion Sort

General strategy:

- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.

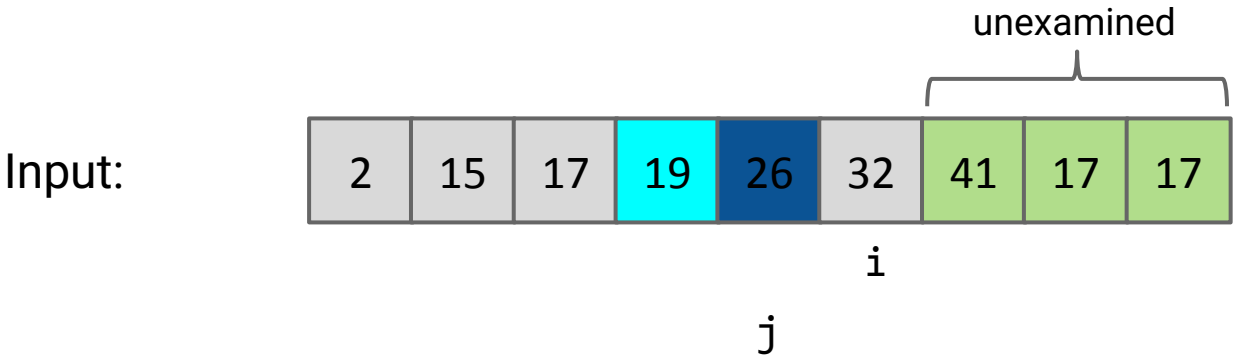


Use `j` pointer to track current spot of traveling item.

In-place Insertion Sort

General strategy:

- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.

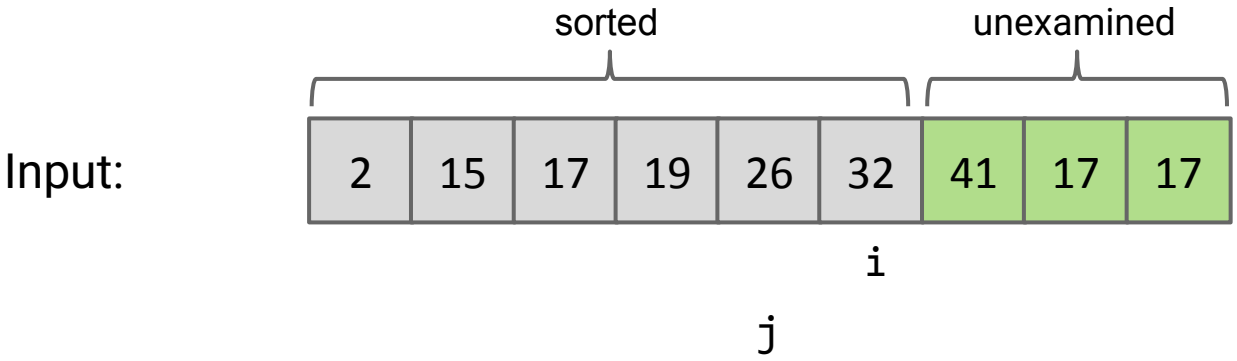


Use `j` pointer to track current spot of traveling item.

In-place Insertion Sort

General strategy:

- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.

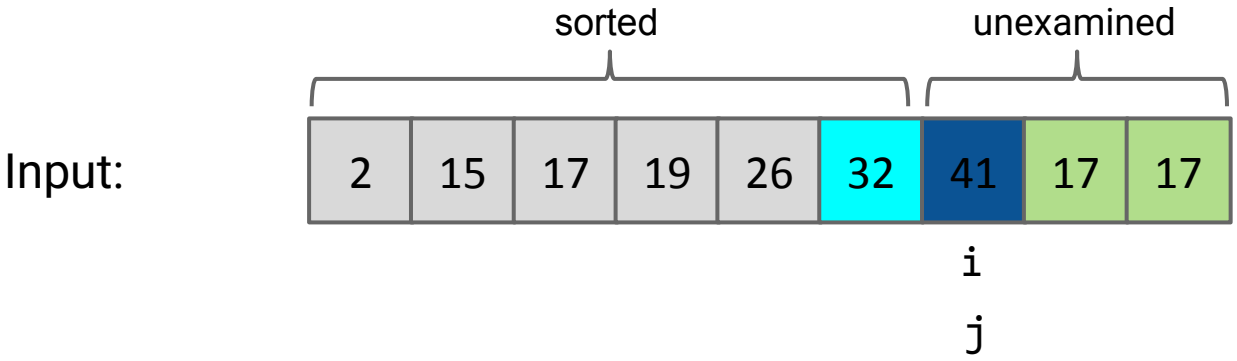


Use `j` pointer to track current spot of traveling item.

In-place Insertion Sort

General strategy:

- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.

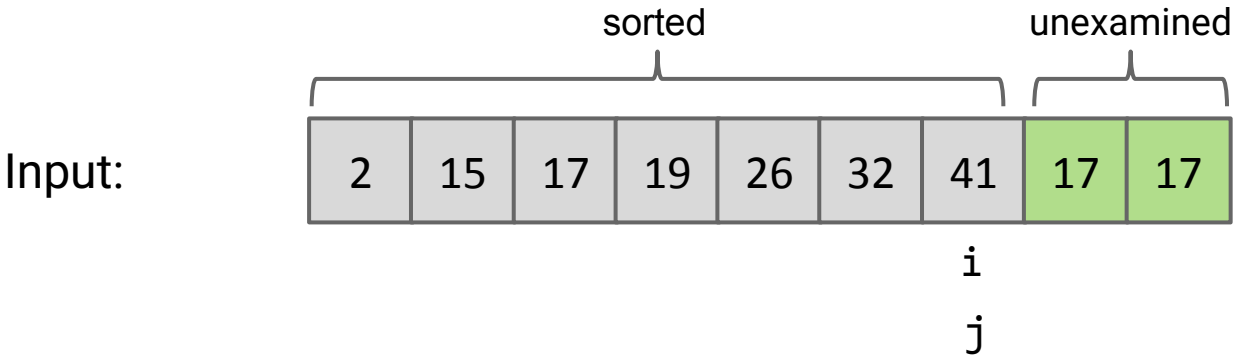


Use `j` pointer to track current spot of traveling item.

In-place Insertion Sort

General strategy:

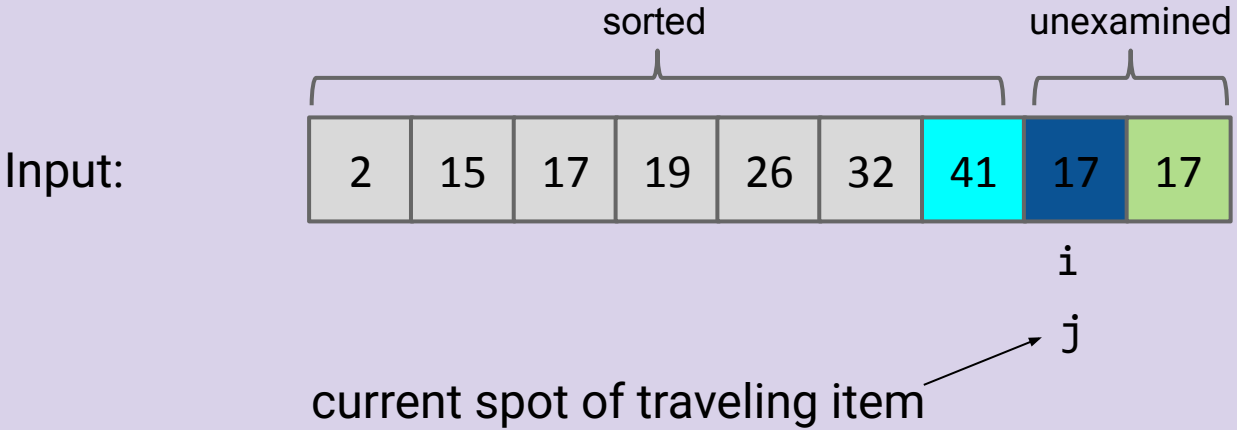
- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.



Use `j` pointer to track current spot of traveling item.

General strategy:

- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.

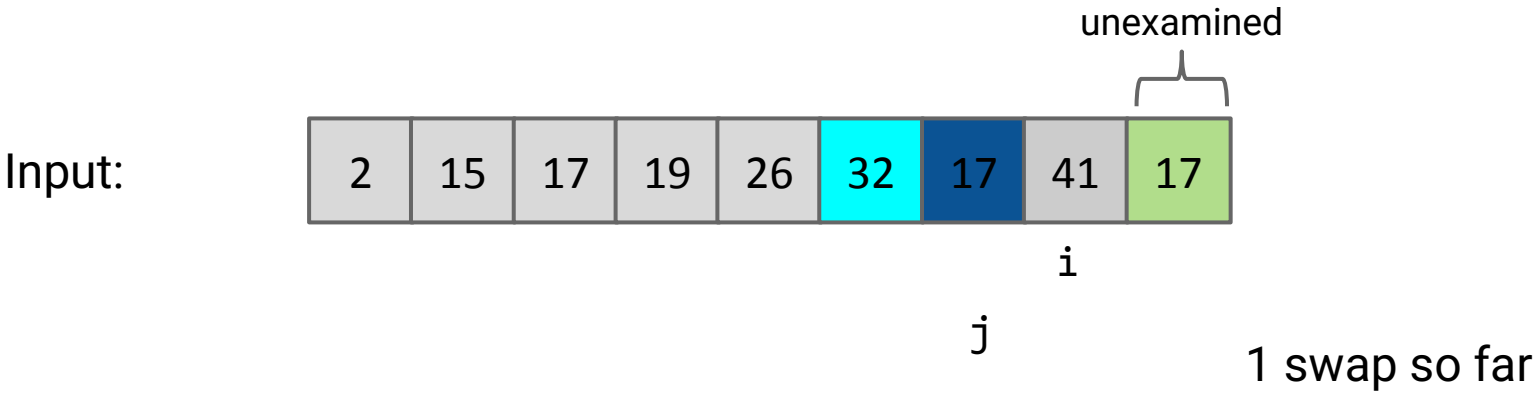


How many swaps do we need to do to get 17 in the correct place?

In-place Insertion Sort

General strategy:

- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.

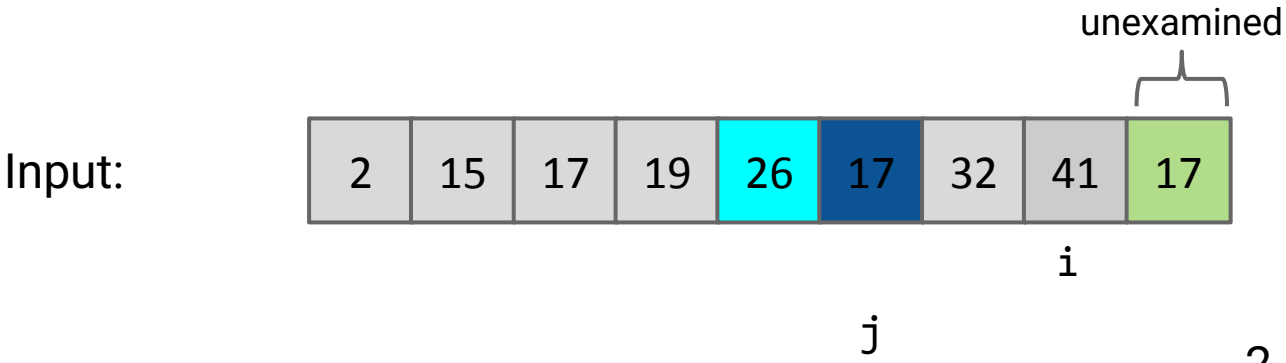


Use `j` pointer to track current spot of traveling item.

In-place Insertion Sort

General strategy:

- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.

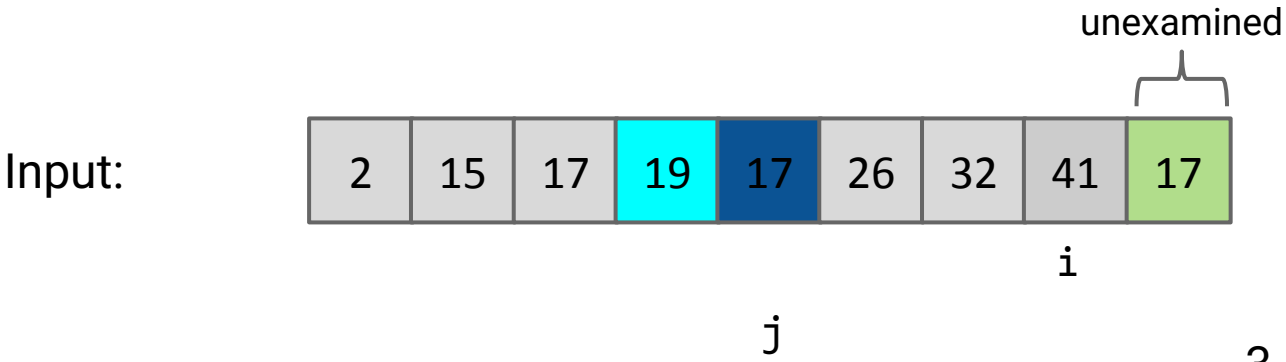


Use `j` pointer to track current spot of traveling item.

In-place Insertion Sort

General strategy:

- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.

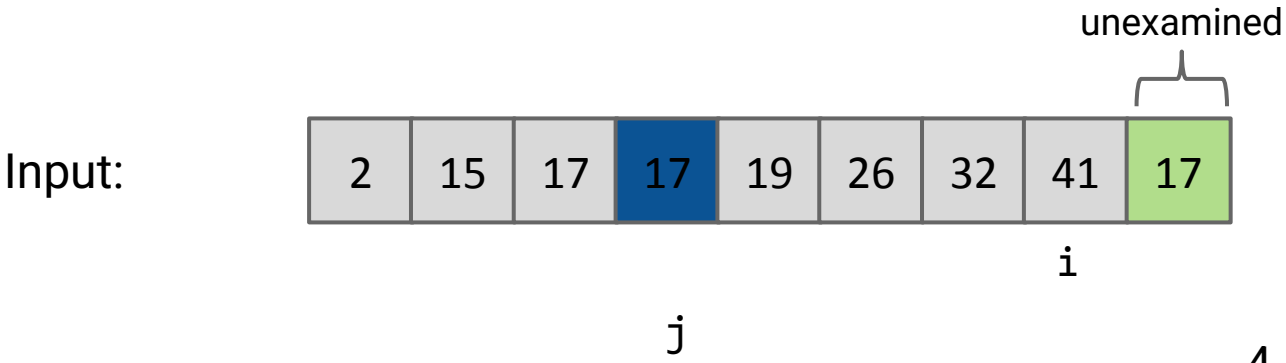


Use `j` pointer to track current spot of traveling item.

In-place Insertion Sort

General strategy:

- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.



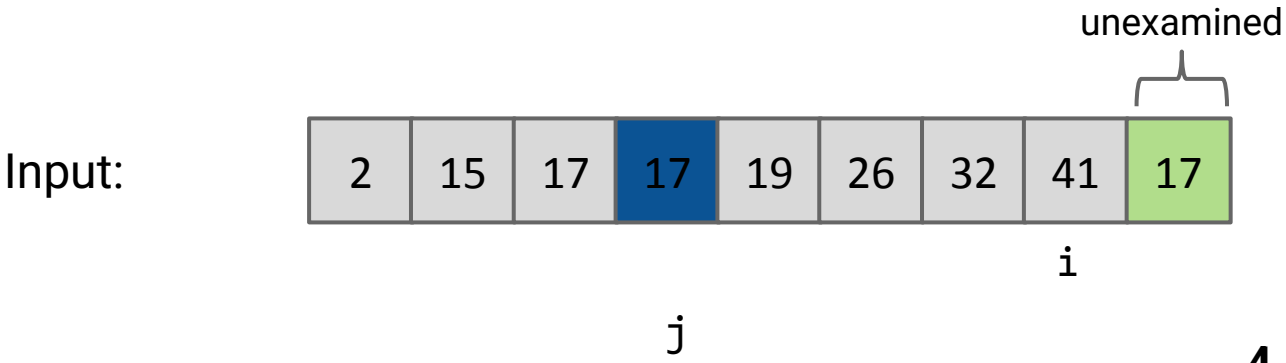
4 swaps so far

Use `j` pointer to track current spot of traveling item.

In-place Insertion Sort

General strategy:

- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.



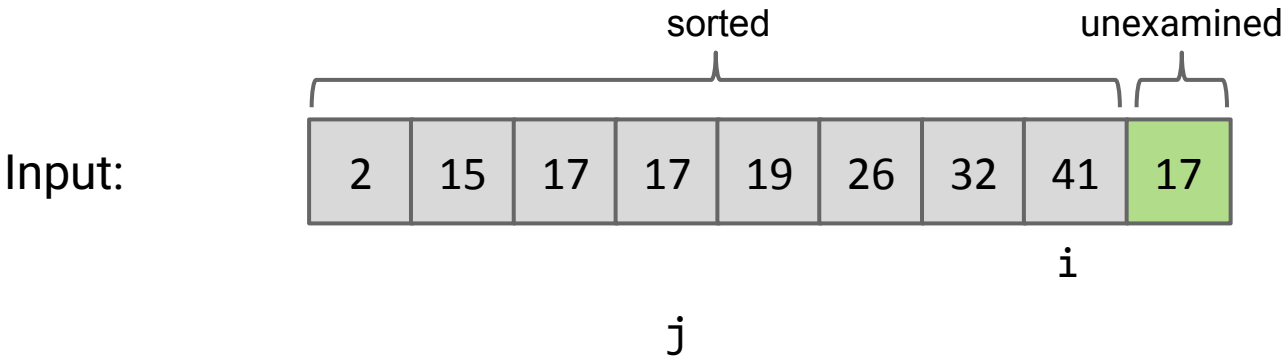
4 swaps total

Use `j` pointer to track current spot of traveling item.

In-place Insertion Sort

General strategy:

- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.

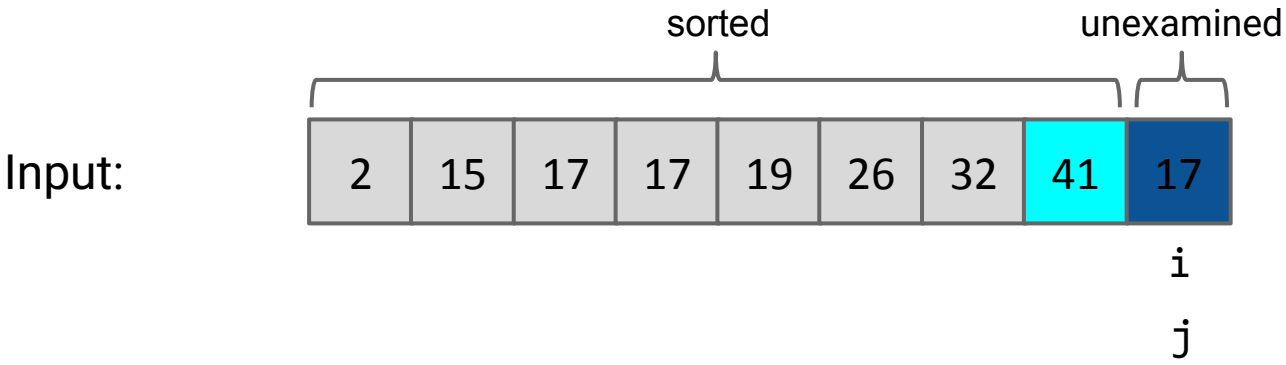


Use `j` pointer to track current spot of traveling item.

In-place Insertion Sort

General strategy:

- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.

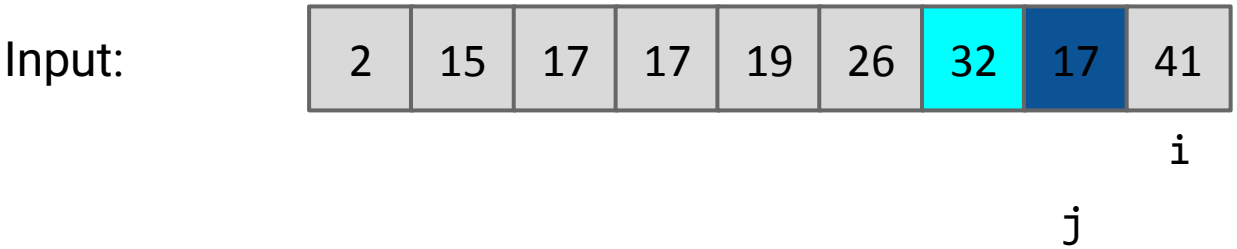


Use `j` pointer to track current spot of traveling item.

In-place Insertion Sort

General strategy:

- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.



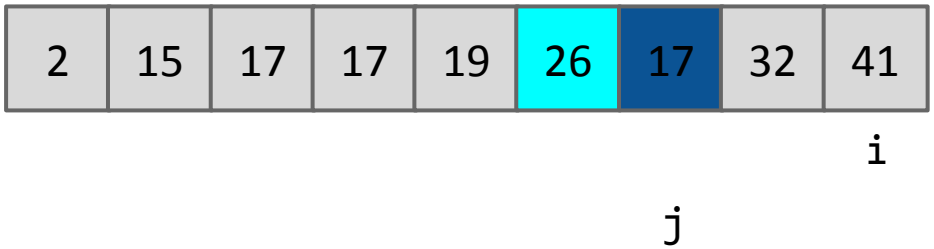
Use `j` pointer to track current spot of traveling item.

In-place Insertion Sort

General strategy:

- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.

Input:

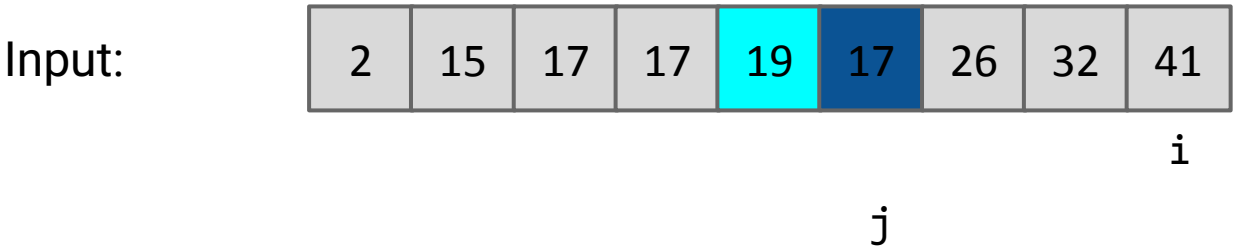


Use `j` pointer to track current spot of traveling item.

In-place Insertion Sort

General strategy:

- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.

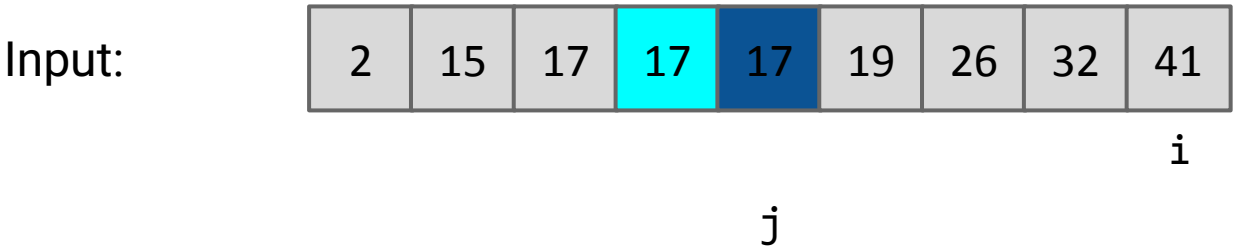


Use `j` pointer to track current spot of traveling item.

In-place Insertion Sort

General strategy:

- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.

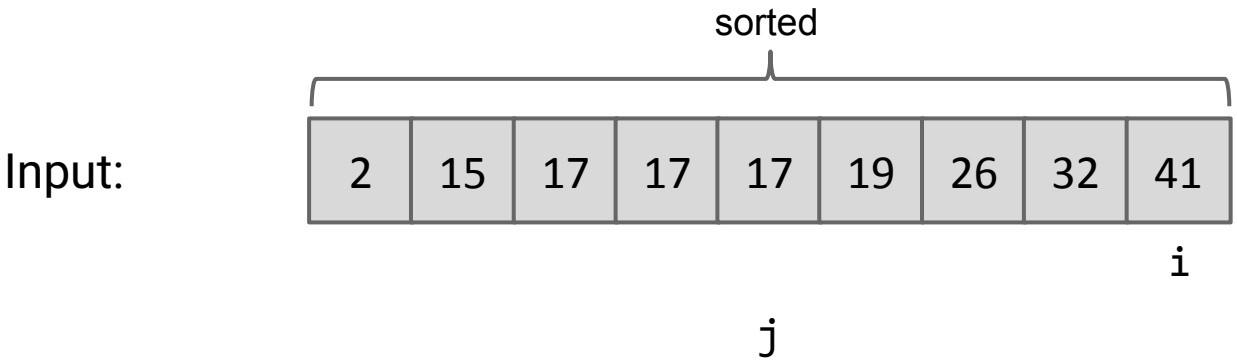


Use `j` pointer to track current spot of traveling item.

In-place Insertion Sort

General strategy:

- Repeat for i = 0 to N - 1:
  - “Insert” item at index i into the right place among sorted items (**traveller**)
    - How? Swap item backwards until traveller is in the right place among all previously examined items.

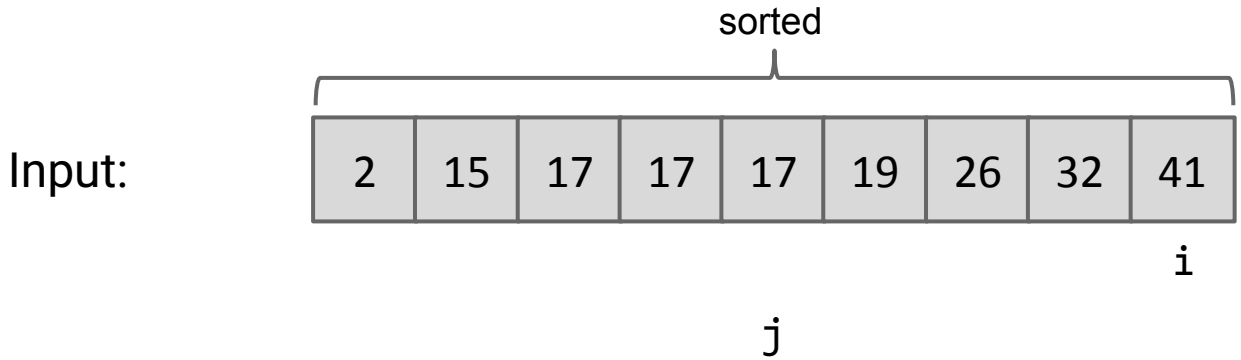


Use `j` pointer to track current spot of traveling item.

Names

From John Ousterhout's Software Design Lecture

- **Good names make code more obvious**
- **Test: if someone sees this name in isolation, will they be able to guess its meaning?**
- **Biggest problem with names: too vague**

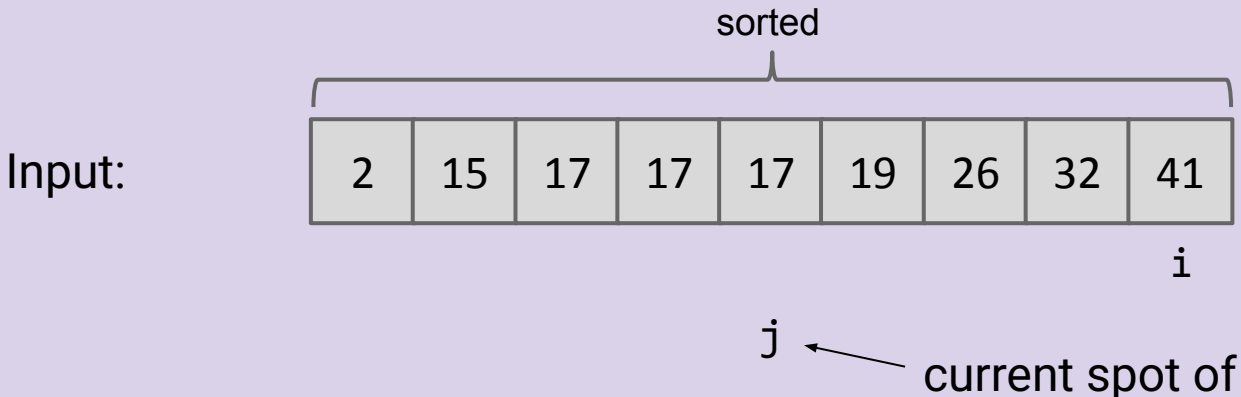


Use `j` pointer to track current spot of traveling item.

What new names would you propose for i and j?

Should we take a more extreme step?

- **If you're having trouble finding a clear and simple name, that's a red flag:**
  - Entity has complex meaning
  - Change the meaning?
  - Divide into two simpler things?
  - Merge with something else?

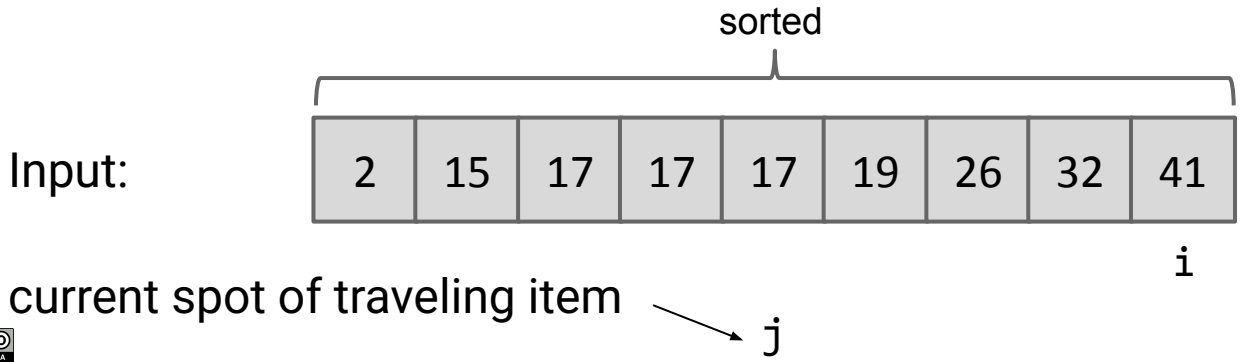


In-place Insertion Sort: Reflections on Naming

What new names would you propose for i and j? Should we take a more extreme step?

Your ideas:

- `J -> travelerl i -> start`
- `i: currentIndex`
- `i: lastSortedIndex, j: traveler`



In-place Insertion Sort: Reflections on Naming

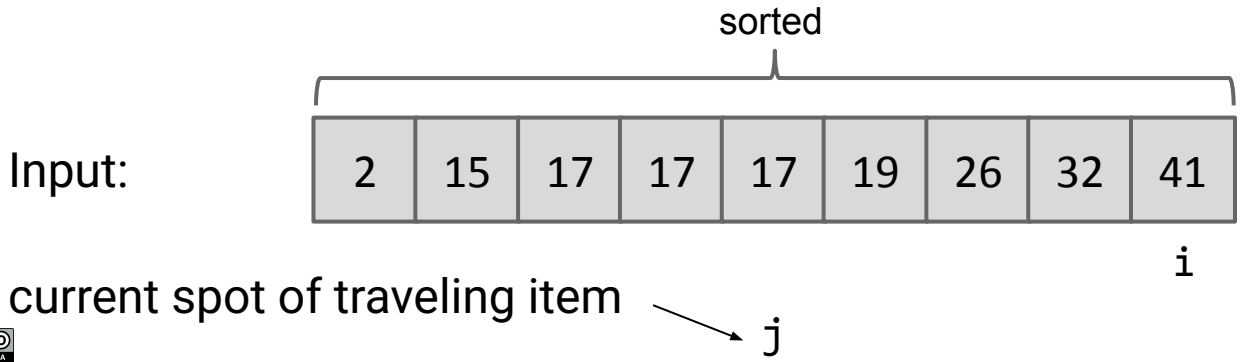
What new names would you propose for i and j? Should we take a more extreme step?

John Ousterhout's suggestions:

- Replace `j` with **`traveller`**
- Awkward to even describe `i`; change to **`examined`** and change semantics to be one past `i`. Comment `// Includes current traveller`

Kay's ideas:

- Replace **`i`** with **`sorted_until`** or **`num_sorted`**



Insertion Sort Runtime

Lecture 33, CS61B, Spring 2026

Insertion Sort

- Naive Insertion Sort
- In-Place Insertion Sort
- **Insertion Sort Runtime**

Miscellaneous Sorts

Quicksort

- Quicksort Backstory, Partitioning
- Quicksort

In-place Insertion Sort

Two more examples.

7 swaps:

```
P O T A T O
P O T A T O  (0 swaps)
O P T A T O  (1 swap )
O P T A T O  (0 swaps)
A O P T T O  (3 swaps)
A O P T T O  (0 swaps)
A O O P T T  (3 swaps)
```

36 swaps:

```
S O R T E X A M P L E
S O R T E X A M P L E  (0 swaps)
O S R T E X A M P L E  (1 swap )
O R S T E X A M P L E  (1 swap )
O R S T E X A M P L E  (0 swaps)
E O R S T X A M P L E  (4 swaps)
E O R S T X A M P L E  (0 swaps)
A E O R S T X M P L E  (6 swaps)
A E M O R S T X P L E  (5 swaps)
A E M O P R S T X L E  (4 swaps)
A E L M O P R S T X E  (7 swaps)
A E E L M O P R S T X  (8 swaps)
```

Purple: Element that we're moving left (with swaps).
Black: Elements that got swapped with purple.
Grey: Not considered this iteration.

Insertion Sort Runtime: yellkey.com/she

What is the best- and worst-case runtime of insertion sort?

A. Best: $\Theta(1)$, Worst: $\Theta(N)$
B. Best: $\Theta(N)$, Worst: $\Theta(N)$
C. Best: $\Theta(1)$, Worst: $\Theta(N^2)$
D. Best: $\Theta(N)$, Worst: $\Theta(N^2)$
E. Best: $\Theta(N^2)$, Worst: $\Theta(N^2)$

36 swaps:

```
S O R T E X A M P L E
S O R T E X A M P L E  (0 swaps)
O S R T E X A M P L E  (1 swap )
O R S T E X A M P L E  (1 swap )
O R S T E X A M P L E  (0 swaps)
E O R S T X A M P L E  (4 swaps)
E O R S T X A M P L E  (0 swaps)
A E O R S T X M P L E  (6 swaps)
A E M O R S T X P L E  (5 swaps)
A E M O P R S T X L E  (4 swaps)
A E L M O P R S T X E  (7 swaps)
A E E L M O P R S T X  (8 swaps)
```

Insertion Sort Runtime

What is the best- and worst-case runtime of insertion sort?

A. Best: $\Theta(1)$, Worst: $\Theta(N)$
B. Best: $\Theta(N)$, Worst: $\Theta(N)$
C. Best: $\Theta(1)$, Worst: $\Theta(N^2)$
D. **Best: $\Theta(N)$, Worst: $\Theta(N^2)$**
E. Best: $\Theta(N^2)$, Worst: $\Theta(N^2)$

36 swaps:

```
S O R T E X A M P L E
S O R T E X A M P L E  (0 swaps)
O S R T E X A M P L E  (1 swap )
O R S T E X A M P L E  (1 swap )
O R S T E X A M P L E  (0 swaps)
E O R S T X A M P L E  (4 swaps)
E O R S T X A M P L E  (0 swaps)
A E O R S T X M P L E  (6 swaps)
A E M O R S T X P L E  (5 swaps)
A E M O P R S T X L E  (4 swaps)
A E L M O P R S T X E  (7 swaps)
A E E L M O P R S T X  (8 swaps)
```

Picking the Best Sort: yellkey.com/notice

Suppose we do the following:

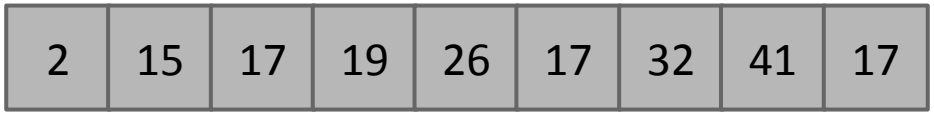
- Start with a sorted array of 1,000,000 integers.
- Select one integer randomly and change it.
- Sort using algorithm X of your choice.

Which sorting algorithm would be the fastest choice for X?

A. Selection Sort: $O(N^2)$
B. Heapsort: O(N Log N)
C. Mergesort: O(N Log N)
D. Insertion Sort: $O(N^2)$

Recall: Inversions

An ***inversion*** is a pair of elements that are out of order with respect to <.



Another way to state the goal of sorting:

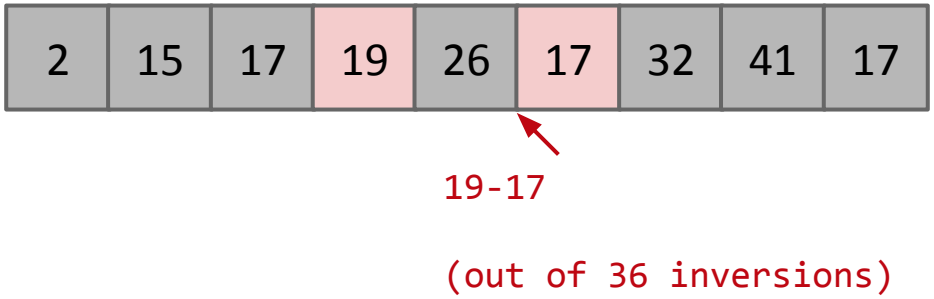
- Given a sequence of elements with Z inversions.
- Perform a sequence of operations that reduces inversions to 0.

What's the worst case number of inversions?

- (N-1) + (N-2) + … + 1 = N * (N - 1) / 2

Recall: Inversions

An ***inversion*** is a pair of elements that are out of order with respect to <.

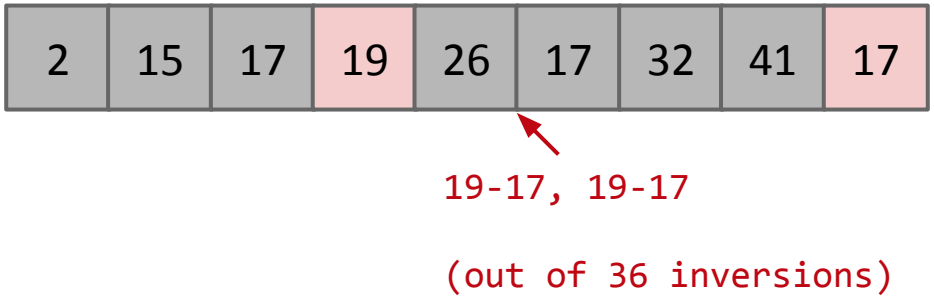


Another way to state the goal of sorting:

- Given a sequence of elements with Z inversions.
- Perform a sequence of operations that reduces inversions to 0.

Recall: Inversions

An ***inversion*** is a pair of elements that are out of order with respect to <.

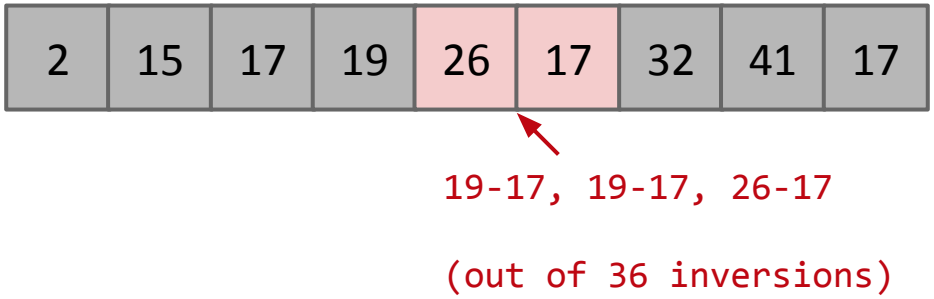


Another way to state the goal of sorting:

- Given a sequence of elements with Z inversions.
- Perform a sequence of operations that reduces inversions to 0.

Recall: Inversions

An ***inversion*** is a pair of elements that are out of order with respect to <.

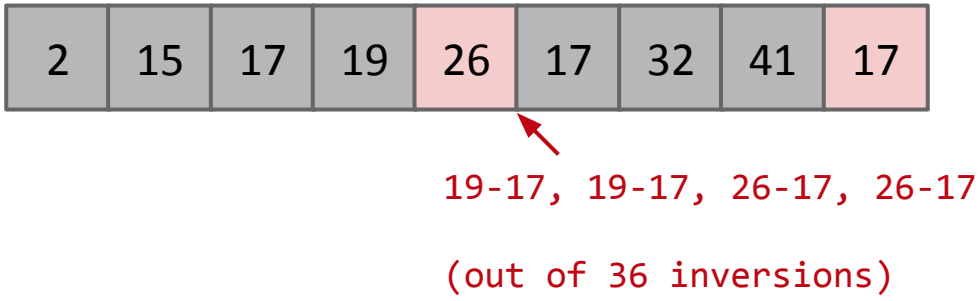


Another way to state the goal of sorting:

- Given a sequence of elements with Z inversions.
- Perform a sequence of operations that reduces inversions to 0.

Recall: Inversions

An ***inversion*** is a pair of elements that are out of order with respect to <.

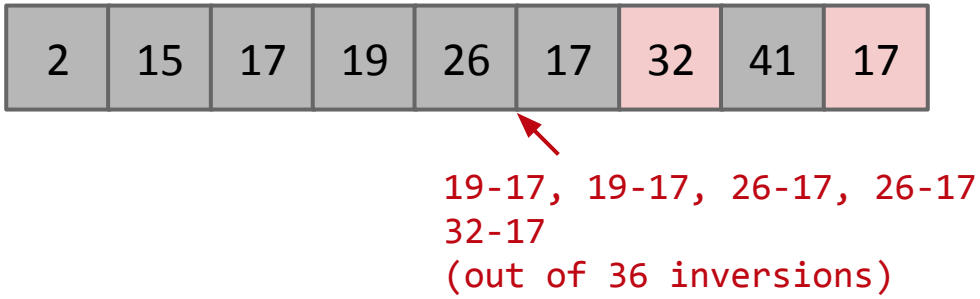


Another way to state the goal of sorting:

- Given a sequence of elements with Z inversions.
- Perform a sequence of operations that reduces inversions to 0.

Recall: Inversions

An ***inversion*** is a pair of elements that are out of order with respect to <.

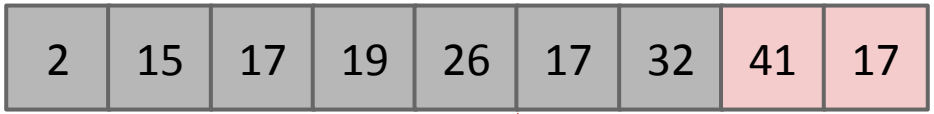


Another way to state the goal of sorting:

- Given a sequence of elements with Z inversions.
- Perform a sequence of operations that reduces inversions to 0.

Recall: Inversions

An ***inversion*** is a pair of elements that are out of order with respect to <.



19-17, 19-17, 26-17, 26-17
32-17, 41-17 = 6 total inversions
(out of 36 inversions)

Another way to state the goal of sorting:

- Given a sequence of elements with Z inversions.
- Perform a sequence of operations that reduces inversions to 0.

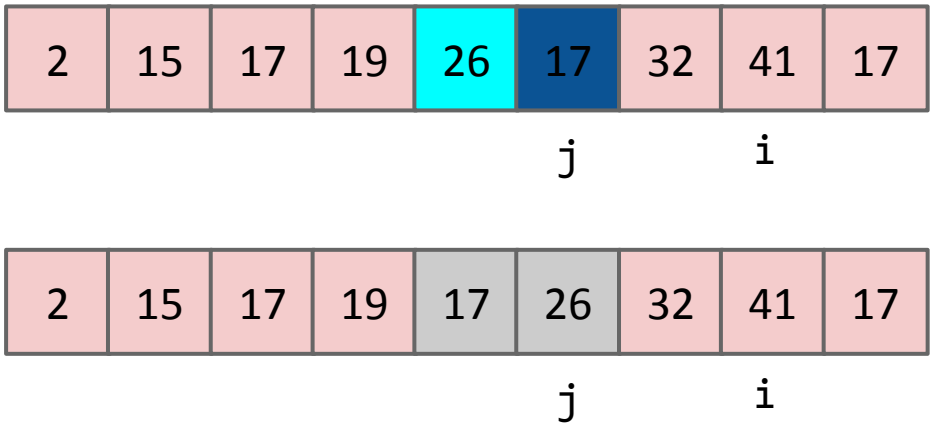
Observation: Each swap in Insertion Sort reduces the number of inversions by 1

Inversions between red items don't change (because they don't move)

Inversions between a red and blue item don't change (their relative position in the array stays the same)

We fix one inversion within the blue items

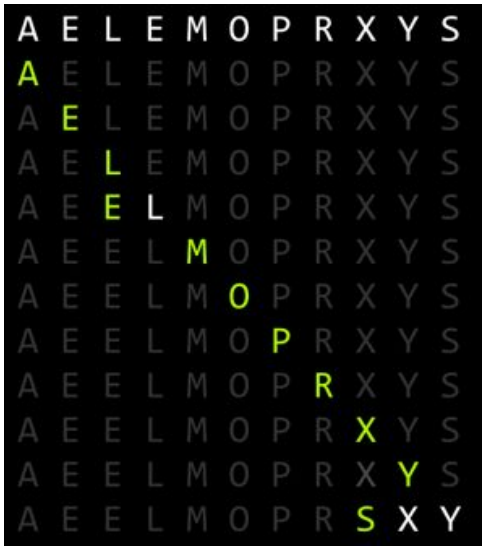
So total number of inversions decrease by 1 every swap



Observation: Insertion Sort on Almost Sorted Arrays

For arrays that are almost sorted, insertion sort does very little work.

- Left array: 5 inversions, so only 5 swaps.
- Right array: 3 inversion, so only 3 swaps.



Picking the Best Sort: How many inversions?

Suppose we do the following:

- Start with a sorted array of 1,000,000 integers.
- Select one integer randomly and change it.
- Sort using algorithm X of your choice.

What's the worst-case number of inversions here?

Example shorter array: [9, 12, 25, 34, 55]

What one number would you change to introduce the most inversions?

Picking the Best Sort: How many inversions?

Suppose we do the following:

- Start with a sorted array of 1,000,000 integers.
- Select one integer randomly and change it.
- Sort using algorithm X of your choice.

What's the worst-case number of inversions here? N - 1

Example shorter array: [9, 12, 25, 34, 55]

What one number would you change to introduce the most inversions?

- Change 9 to 56 (4 inversions; ever item after it)
- Change 55 to 8 (inverted with every item before it)

Picking the Best Sort

Suppose we do the following:

- Start with a sorted array of 1,000,000 integers.
- Select one integer randomly and change it.
- Sort using algorithm X of your choice.

In the worst case, we have N - 1 inversions (Θ(N) inversions)

Which sorting algorithm would be the fastest choice for X? Worst case run-times:

A. Selection Sort: $O(N^2)$
B. Heapsort: O(N log N)
C. Mergesort: O(N log N)
D. Insertion Sort $O(N^2)$

Picking the Best Sort

Suppose we do the following:

- Start with a sorted array of 1,000,000 integers.
- Select one integer randomly and change it.
- Sort using algorithm X of your choice.

In the worst case, we have N - 1 inversions (Θ(N) inversions)

Which sorting algorithm would be the fastest choice for X? Run-times:

A. Selection Sort: $\Theta(N^2)$
B. Heapsort: O(N log N)
C. Mergesort: Θ(N log N)
D. Insertion Sort: ~~$O(N^2)$~~ Θ(N)

Number of swaps = number of inversions!

Insertion Sort Sweet Spots

On arrays with a small number of inversions, insertion sort is extremely fast.

- One exchange per inversion (and number of comparisons is similar). Runtime is $\Theta(N + K)$ where K is number of inversions.
- K is on average $\Theta(N^2)$ in random lists, but…
- Define an ***almost sorted*** array as one in which number of inversions ≤ cN for some c. Insertion sort is excellent on these arrays.

Less obvious: For small arrays (N < 15 or so), insertion sort is fastest.

- More of an empirical fact than a theoretical one.
- Theoretical analysis beyond scope of the course.
- Rough idea: Divide and conquer algorithms like heapsort / mergesort spend too much time dividing, but insertion sort goes straight to the conquest.
- The Java implementation of Mergesort does this (Link).

Sorts So Far

	Best Case Runtime	Worst Case Runtime	Space	Notes
Selection Sort	$\Theta(N^2)$ Find smallest item in remaining items, N times	$\Theta(N^2)$	$\Theta(1)$	
Heapsort (in place)	$\Theta(N)$ All duplicates	$\Theta(N \log N)$	$\Theta(1)$**	Bad cache (61C) performance.
Mergesort	$\Theta(N \log N)$	$\Theta(N \log N)$	$\Theta(N)$	Faster than heapsort.
Insertion Sort (in place)	$\Theta(N)$ cN Inversions	$\Theta(N^2)$	$\Theta(1)$	Best for small N or almost sorted.

See this link for bonus slides on Shell's Sort, an optimization of insertion sort.

Quicksort Backstory, Partitioning

Lecture 33, CS61B, Spring 2026

Insertion Sort

- Naive Insertion Sort
- In-Place Insertion Sort
- Insertion Sort Runtime

Quicksort

- **Quicksort Backstory, Partitioning**
- Quicksort

Sorting So Far

Core ideas:

- Selection sort: Find the smallest item and put it at the front.
  - Heapsort variant: Use MaxPQ to find max element and put at the back.
- Merge sort: Merge two sorted halves into one sorted whole.
- Insertion sort: Figure out where to insert the current item.

Quicksort:

- Core idea: Partitioning.
- Invented by Sir Tony Hoare in 1960, at the time a novice programmer.

Context for Quicksort's Invention (Source)

1960: Tony Hoare was working on a crude automated translation program for Russian and English.

"The cat wore a beautiful hat."

N words

...	...
beautiful	красивая
...	...
cat	кошка
...	...

Dictionary of D english words

"Кошка носил красивая шапка."

How would you do this?

- Binary search for each word.
  - Find "the" in log D time.
  - Find "cat" in log D time...
- Total time: N log D

Context for Quicksort's Invention (Source)

1960: Tony Hoare was working on a crude automated translation program for Russian and English.

Algorithm: N binary searches of D length dictionary.

- Total runtime: N log D
- ASSUMES log time binary search!

...	...
beautiful	красивая
...	...
cat	кошка
...	...

Limitation at the time:

- Dictionary stored on long piece of tape, sentence is an array in RAM.
  - Binary search of tape is not log time (requires physical movement!).
- Better: **Sort the sentence** and scan dictionary tape once. Takes N log N + D time.
  - But Tony had to figure out how to sort an array (without Google!)...

Partitioning

Main idea: I have a list of small and large items. To sort this:

- Separate the small items and large items into two separate piles
- Sort the small items
- Sort the large items
- Put the sorted lists together

How to decide what's big and what's small?

- Pick an item from the list (we'll call this the pivot). Smaller items are small, and bigger items are big

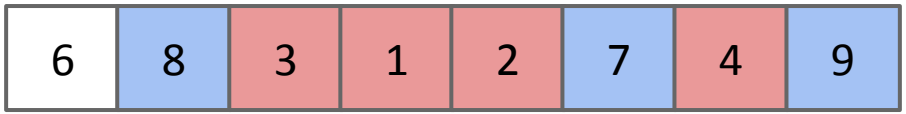
We'll call this a **partition**; we separate the big items and the small items, and put remaining items in the middle.

Partitioning

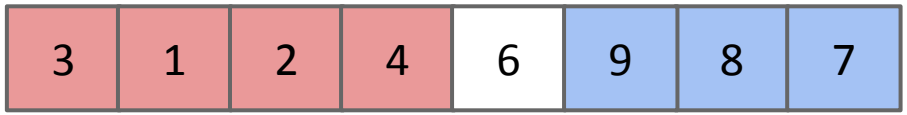
A **partition** of an array, given a **pivot** x, is a rearrangement of the items so that:

- All entries to the left of x are <= x.
- All entries to the right of x are >= x.
- x moves between the smaller and larger items

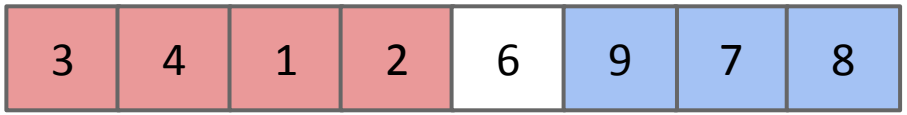
Input



Example of a valid output



Another example of a valid output



Core Idea of Tony's Sort: Partitioning http://yellkey.com/TODO

A **partition** of an array, given a **pivot** x, is a rearrangement of the items so that:

- All entries to the left of x are <= x.
- All entries to the right of x are >= x.

5	550	10	4	10	9	330

A.

			j			
4	5	9	10	10	330	550

B.

			j			
5	4	9	10	10	550	330

C.

				j		
5	9	10	4	10	550	330

D.

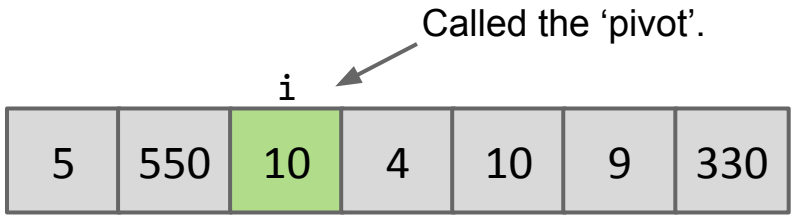
		j				
5	9	10	4	10	550	330

Which partitions are valid?

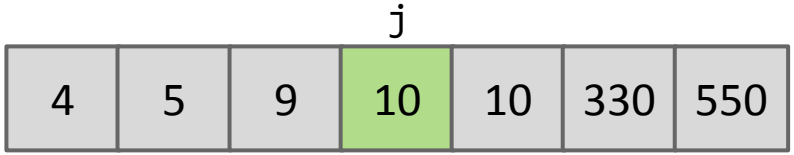
The Core Idea of Tony's Sort: Partitioning

A **partition** of an array, given a **pivot** x, is a rearrangement of the items so that:

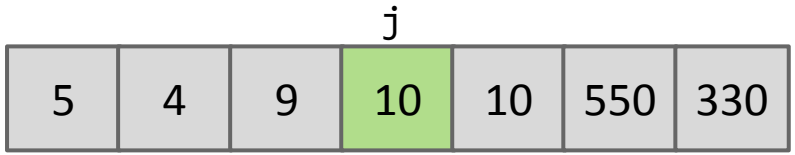
- All entries to the left of x are <= x.
- All entries to the right of x are >= x.



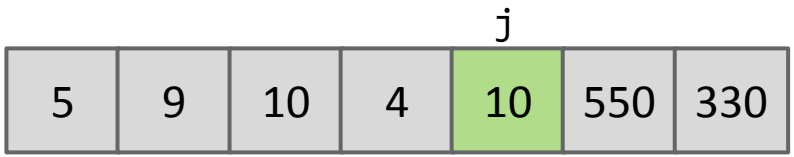
A.



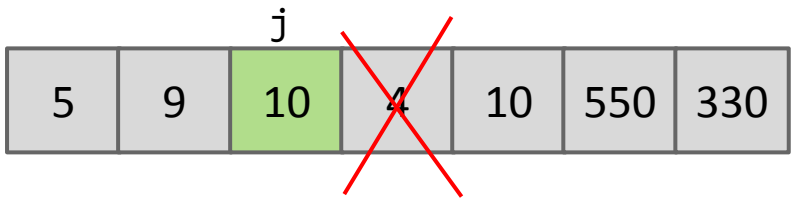
B.



C.



D.



Which partitions are valid?

Job Interview Style Question (Partitioning)

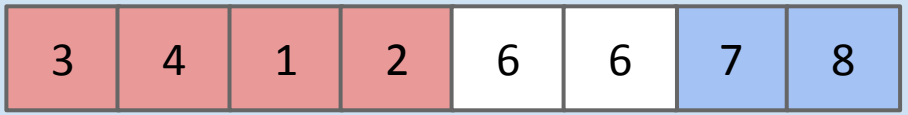
Given an array of colors where the 0th element is white (and maybe a few more), and the remaining elements are red (less) or blue (greater), rearrange the array so that all red squares are to the left of the white square, the white squares end up together, and all blue squares are to the right. Your algorithm must complete in Θ(N) time (no space restriction).

- Relative order of red and blues does NOT need to stay the same.

Input

Example of a valid output

Another example of a valid output



Quicksort

Lecture 33, CS61B, Spring 2026

Insertion Sort

- Naive Insertion Sort
- In-Place Insertion Sort
- Insertion Sort Runtime

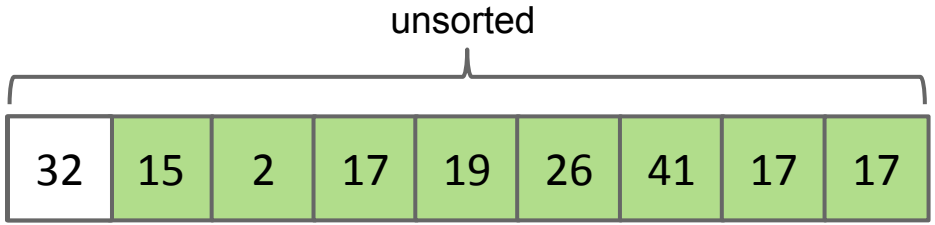
Miscellaneous Sorts

Quicksort

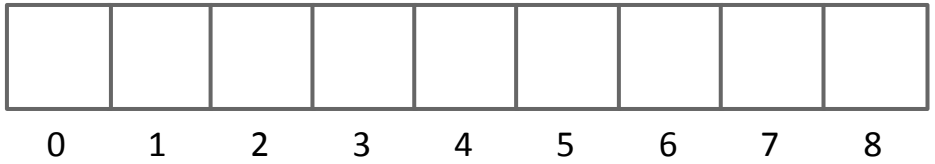
- Quicksort Backstory, Partitioning
- **Quicksort**

Quick Sort: Intuition

Input:

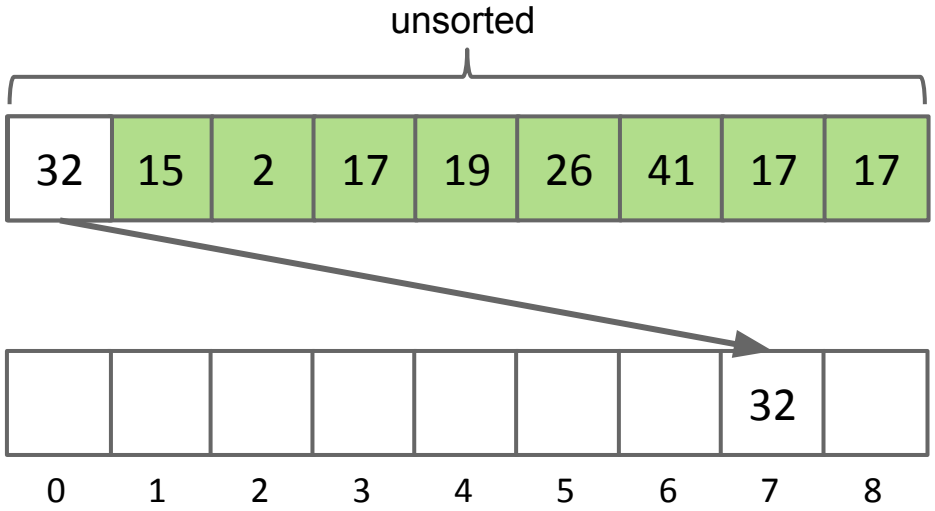


Pick one item ("pivot"), get it in the correct spot



Quick Sort: Intuition

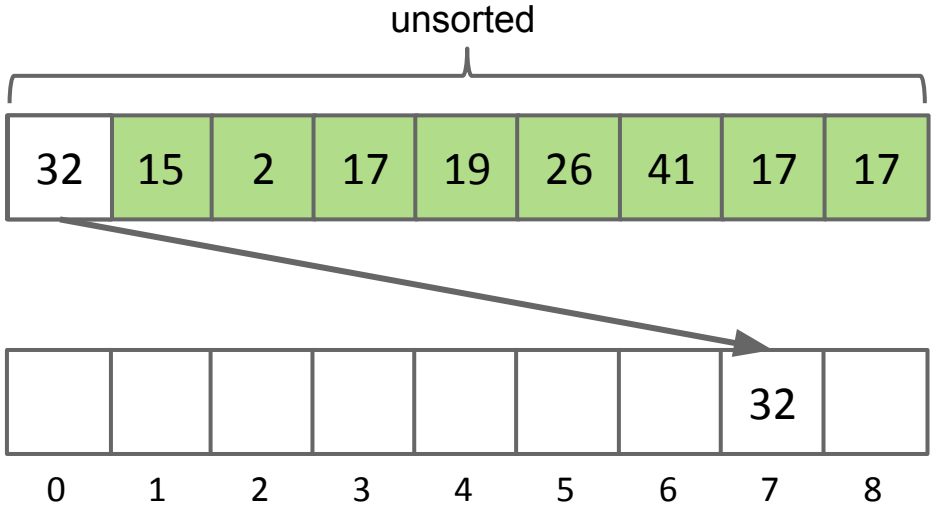
Input:



Pick one item ("pivot"), get it in the correct spot

Quick Sort: Intuition

Input:

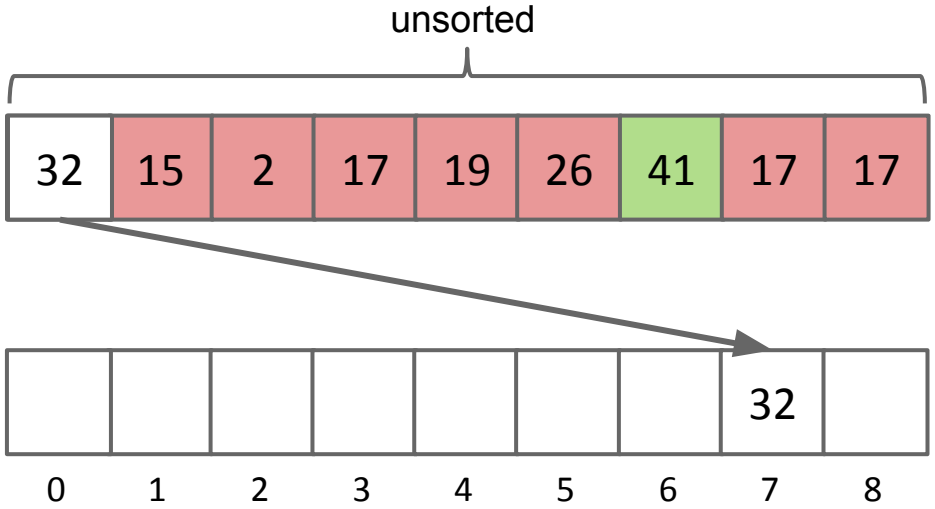


Pick one item ("pivot"), get it in the correct spot

Get everything else on the correct side

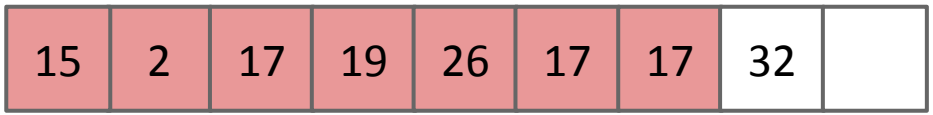
Quick Sort: Intuition

Input:



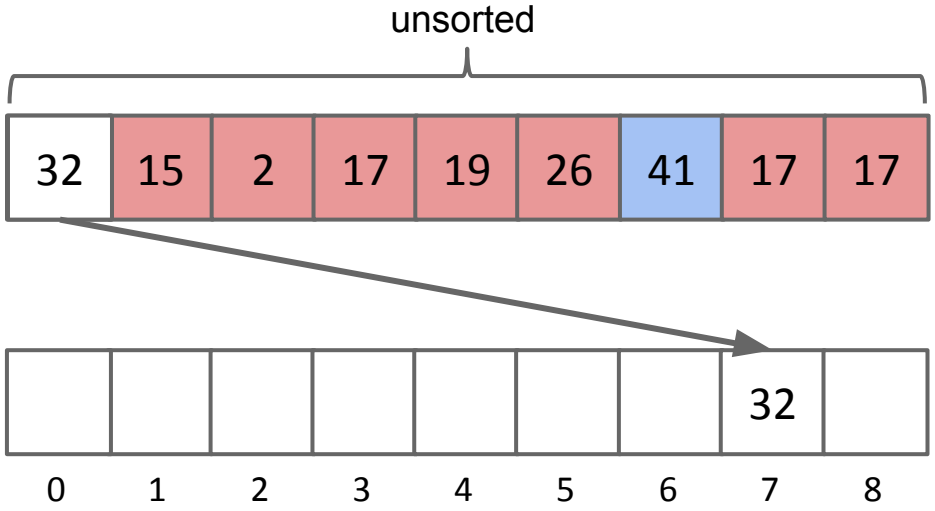
Pick one item ("pivot"), get it in the correct spot

Get everything else on the correct side



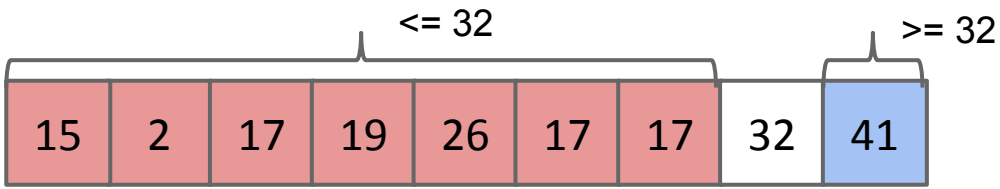
Quick Sort: Intuition

Input:



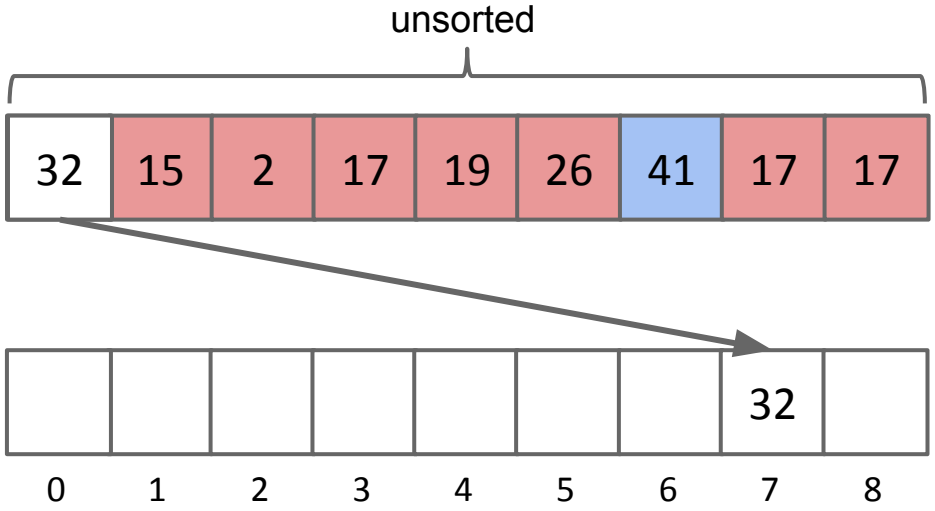
Pick one item ("pivot"), get it in the correct spot

Get everything else on the correct side



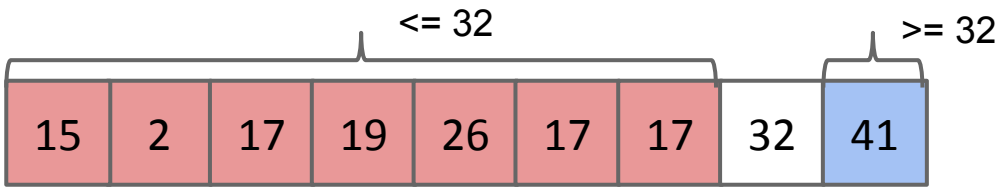
Quick Sort: Intuition

Input:



Pick one item ("pivot"), get it in the correct spot

Get everything else on the correct side



Do this to each item => sorted array

Each time have to look through fewer items => quick

Core Idea of Tony's Sort: Partitioning http://yellkey.com/TODO

A **partition** of an array, given a **pivot** x, is a rearrangement of the items so that:

- All entries to the left of x are <= x.
- All entries to the right of x are >= x.

5	550	10	4	10	9	330

A.

			j			
4	5	9	10	10	330	550

B.

			j			
5	4	9	10	10	550	330

C.

				j		
5	9	10	4	10	550	330

D.

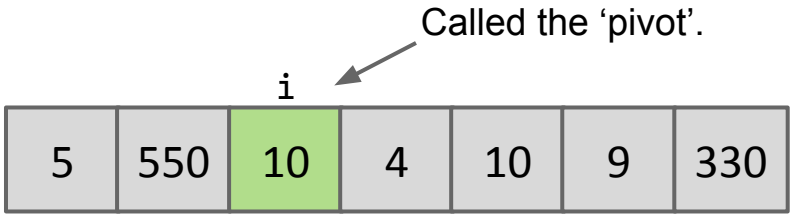
		j				
5	9	10	4	10	550	330

Which partitions are valid?

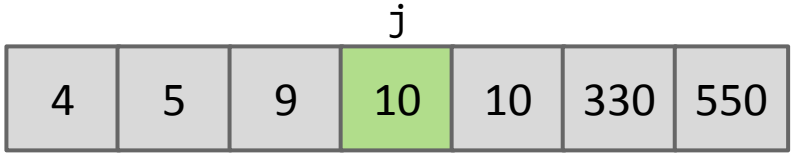
The Core Idea of Tony's Sort: Partitioning

A **partition** of an array, given a **pivot** x, is a rearrangement of the items so that:

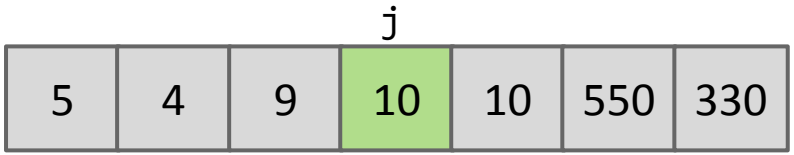
- All entries to the left of x are <= x.
- All entries to the right of x are >= x.



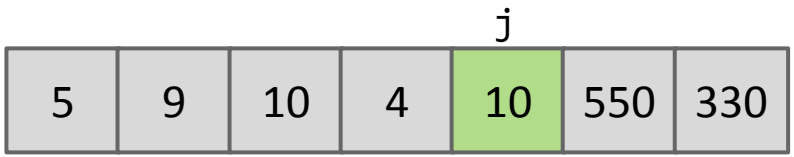
A.



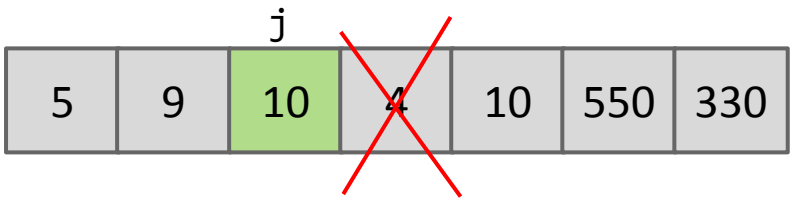
B.



C.

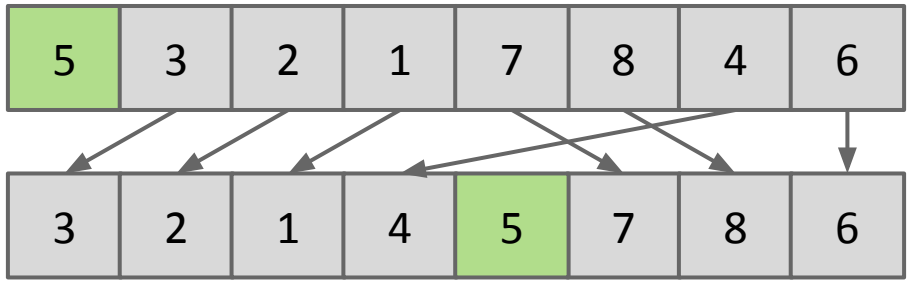


D.



Which partitions are valid?

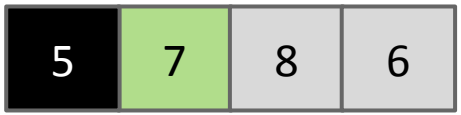
Partition Sort, a.k.a. Quicksort

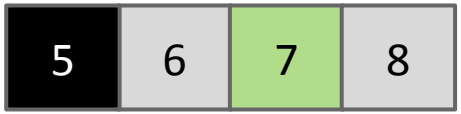


Q: How would we use this operation for sorting?

Observations:

- 5 is “in its place.” Exactly where it’d be if the array were sorted.
- Can sort two halves separately, e.g. through recursive use of partitioning.

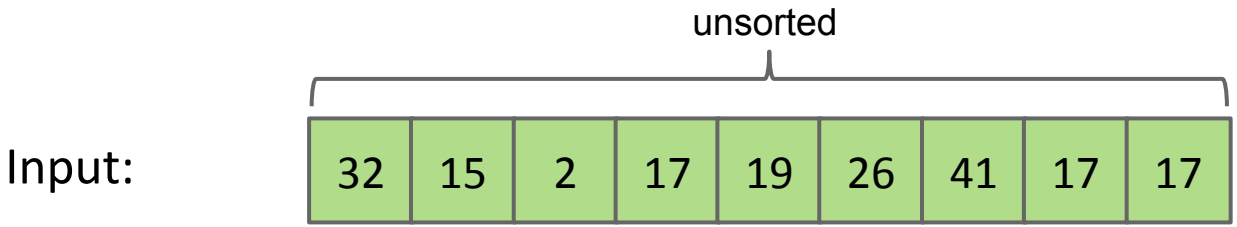




Quick Sort

Quick sorting N items:

- Partition on leftmost item.
- Quicksort left half.
- Quicksort right half.



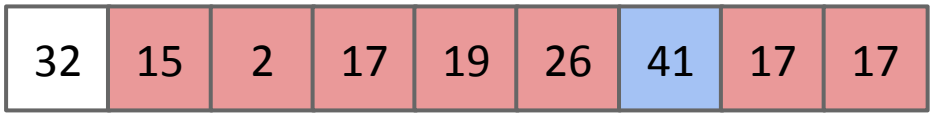
Quick Sort

Quick sorting N items:

- **Partition on leftmost item (32).**
- Quicksort left half.
- Quicksort right half.

partition(32)

Input:

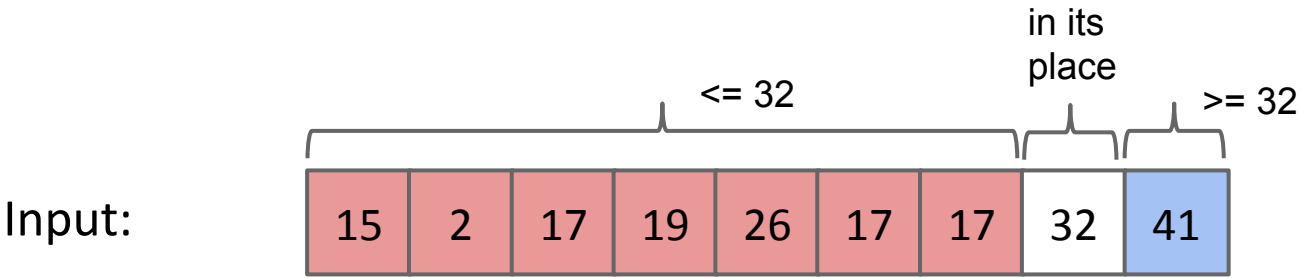


Quick Sort

Quick sorting N items:

partition(32)

- **Partition on leftmost item (32).**
- Quicksort left half.
- Quicksort right half.

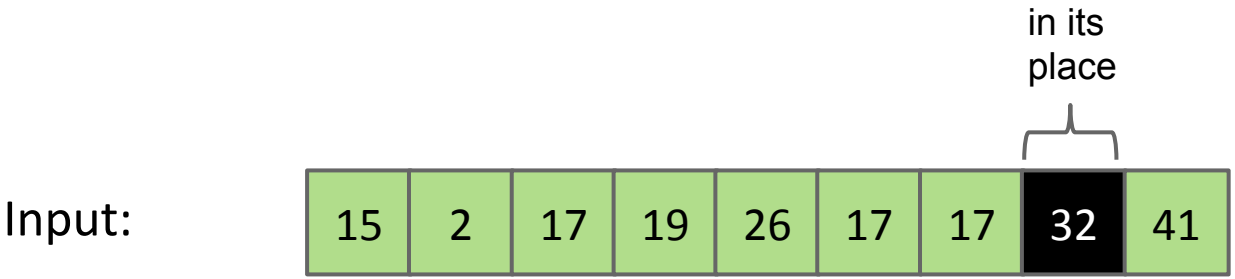


Quick Sort

Quick sorting N items:

partition(32)

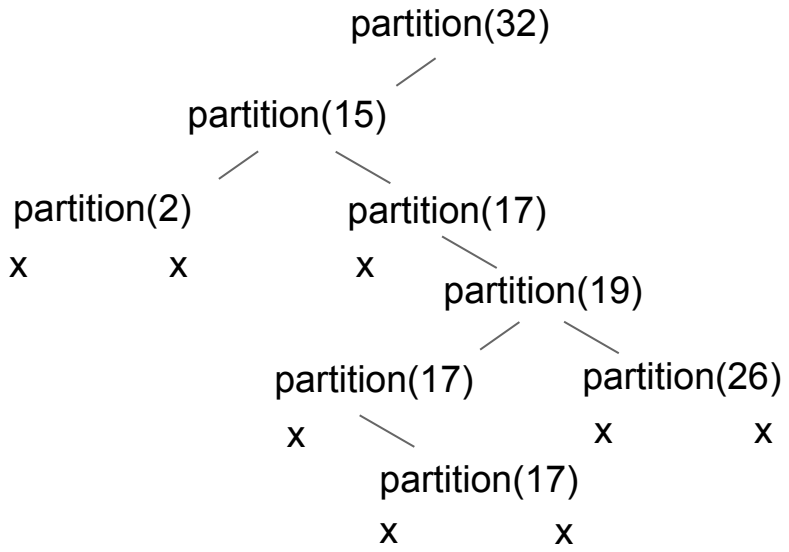
- **Partition on leftmost item (32) (done).**
- Quicksort left half.
- Quicksort right half.

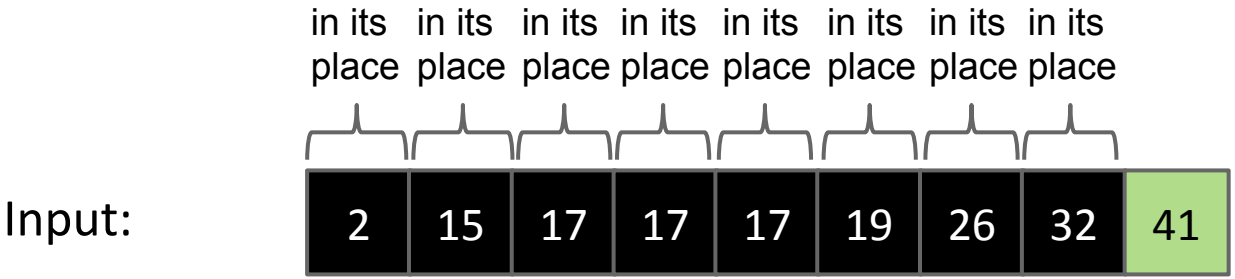


Quick Sort

Quick sorting N items:

- Partition on leftmost item (32) (done).
- **Quicksort left half (details not shown).**
- Quicksort right half.

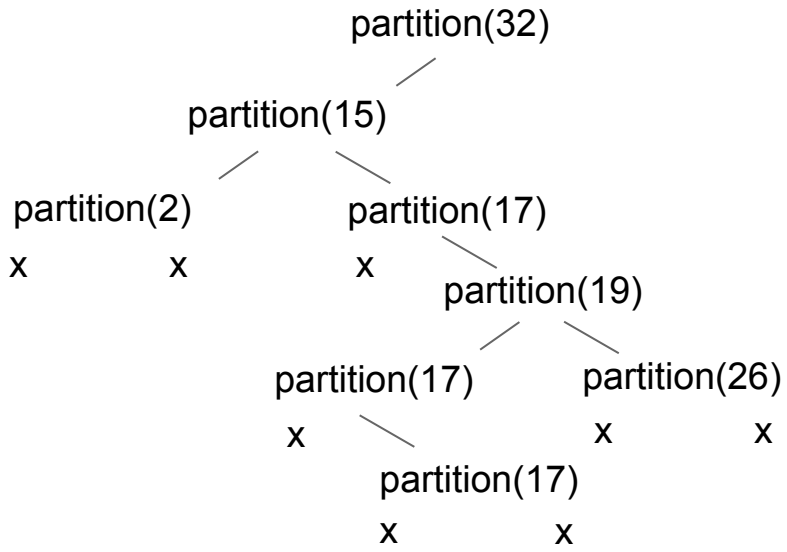




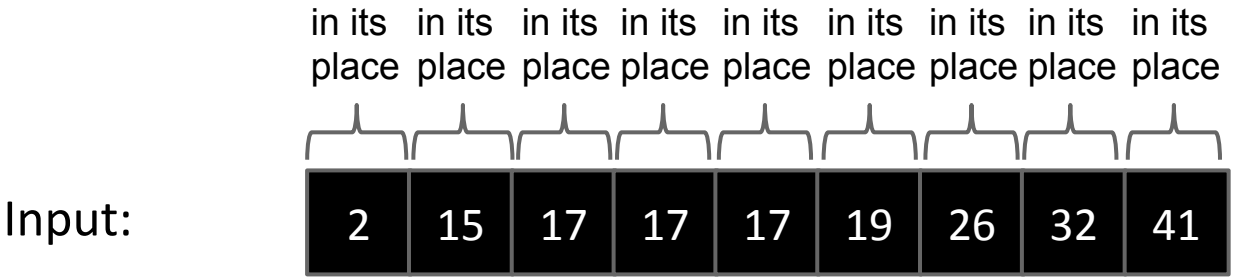
Quick Sort

Quick sorting N items:

- Partition on leftmost item (32) (done).
- Quicksort left half (details not shown).
- **Quicksort right half (details not shown).**



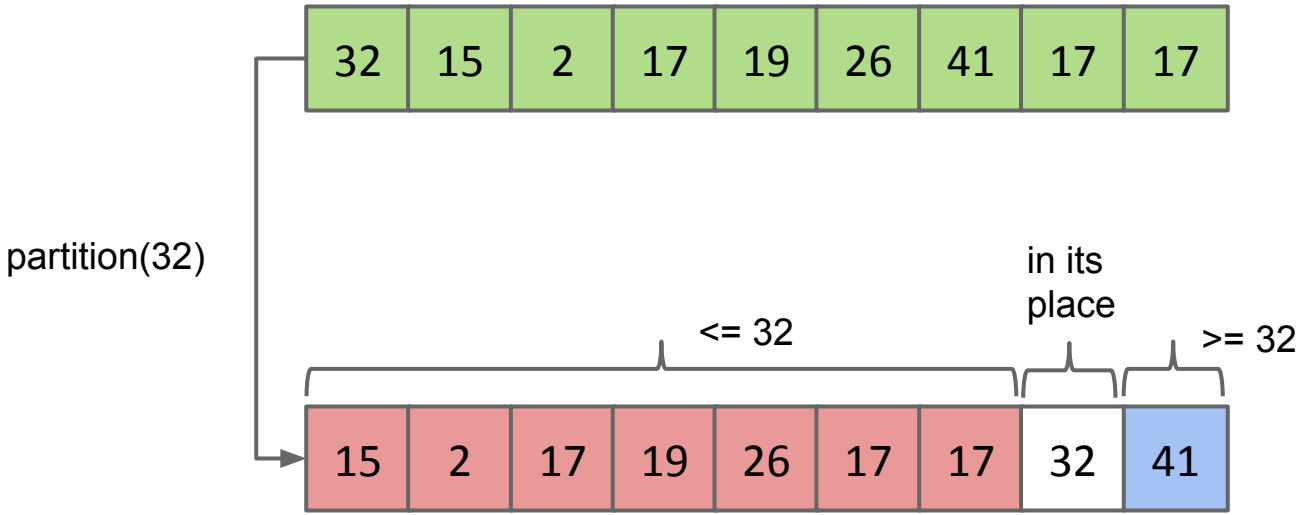
If you don't fully trust the recursion, see these extra slides for a complete demo.



Partition Sort, a.k.a. Quicksort

Quick sorting N items:

- Partition on leftmost item.
- Quicksort left half.
- Quicksort right half.



Quicksort

Quicksort was the name chosen by Tony Hoare for partition sort.

- For most common situations, it is empirically the fastest sort.
  - Tony was lucky that the name was correct.

How fast is Quicksort? Need to count number and difficulty of partition operations.

Theoretical analysis:

- Partitioning costs Θ(K) time, where Θ(K) is the number of elements being partitioned (as we saw in our earlier “interview question”).
- The interesting twist: Overall runtime will depend crucially on where pivot ends up.